

Lecture 5

Sequence Modeling

Speaker:
Kaiming He

Overview

- Introduction to Sequence Modeling
- CNNs for Sequences
- Recurrent Neural Networks (RNNs)
- Memory Mechanisms

Sequence Modeling: Scenarios

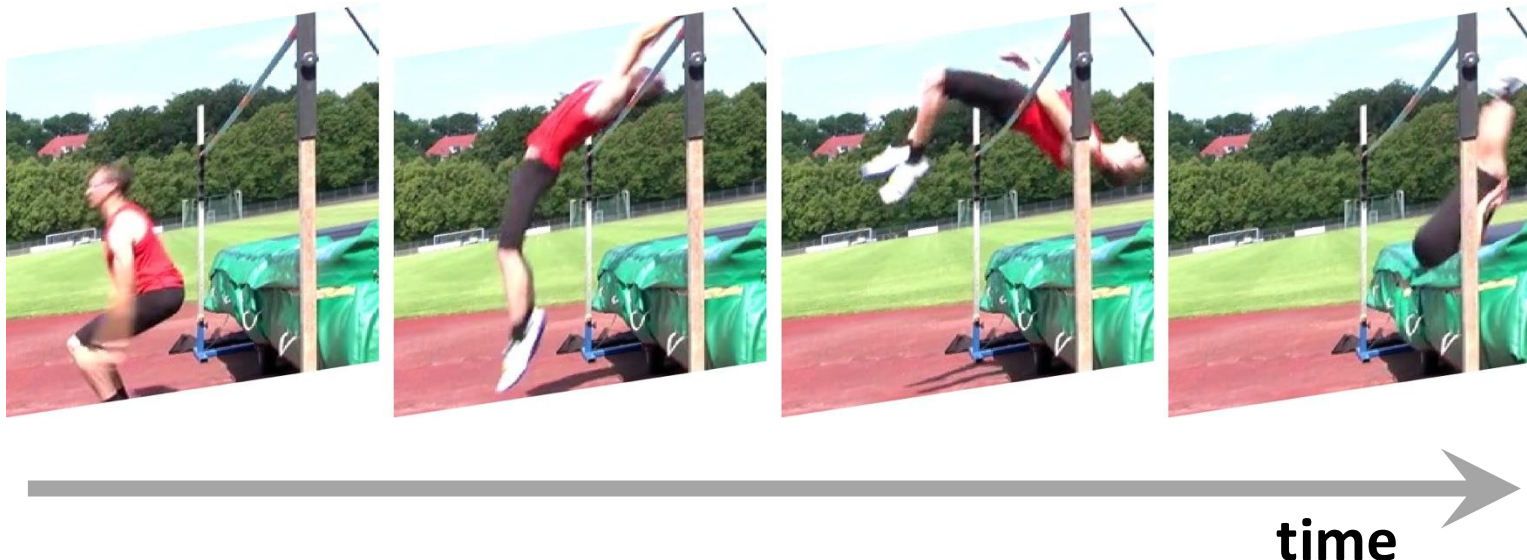
Text

The cat watched the rain through the window.

Audio

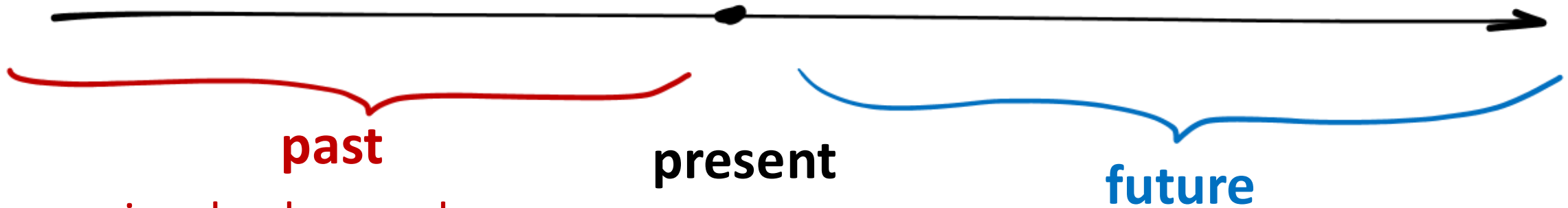


Video



What Makes (Causal) Sequence Modeling Special?

- The Arrow of Time: 3 regimes

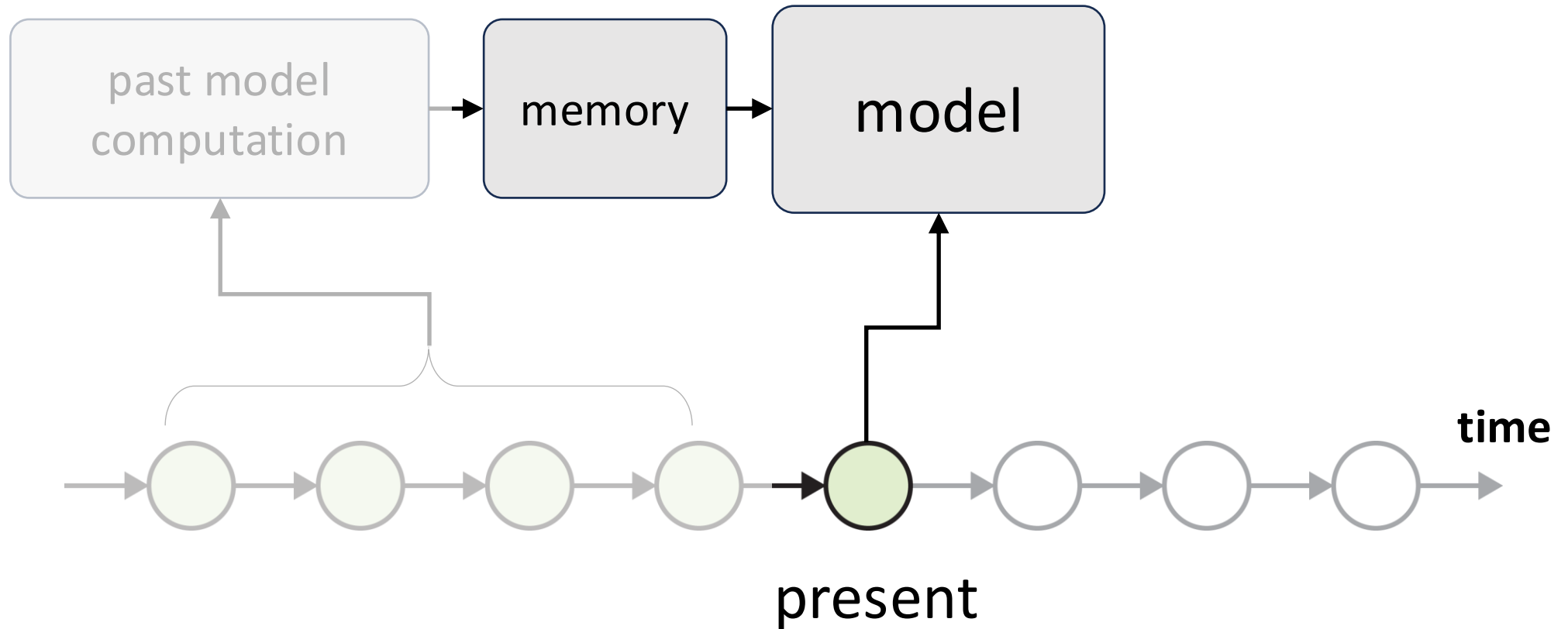


- previously observed
- may be retained in memory
- can be arbitrarily long

- current input to model
- may produce output
- may update memory
- will become “past”

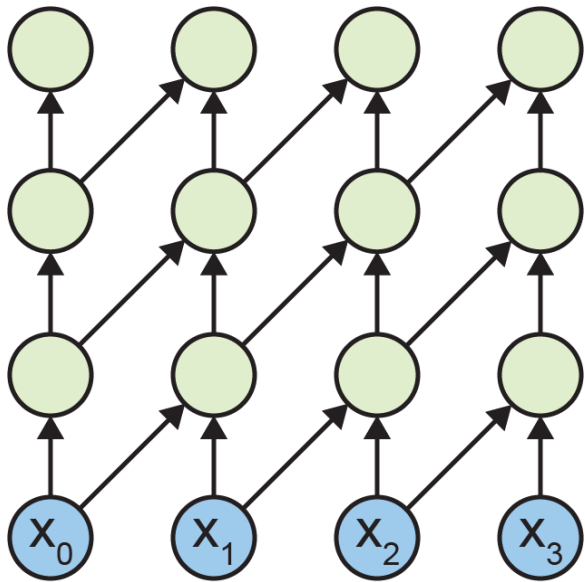
- not yet observed
- may be predicted
- will become “present”

A General View of Sequence Modeling

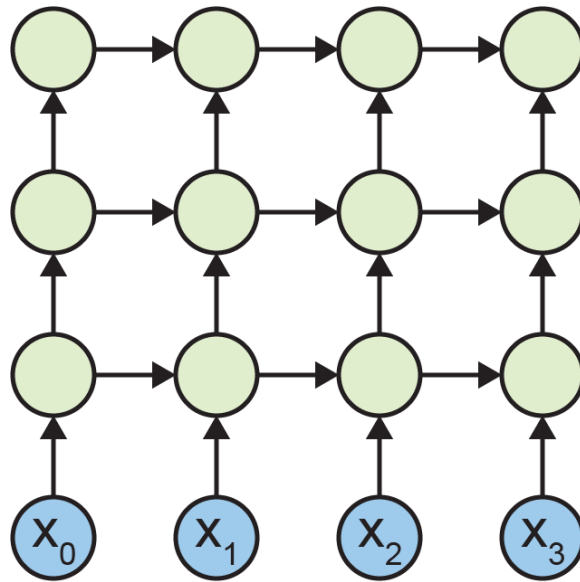


- The model only sees present & memory

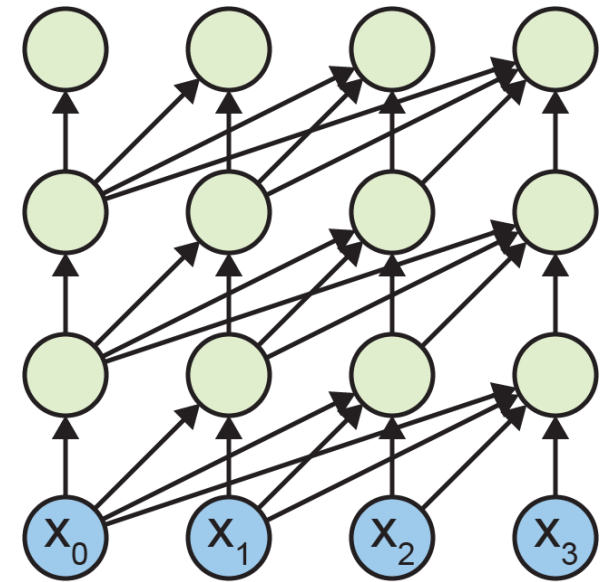
Paradigms of Sequence Modeling



CNN



RNN



Attention

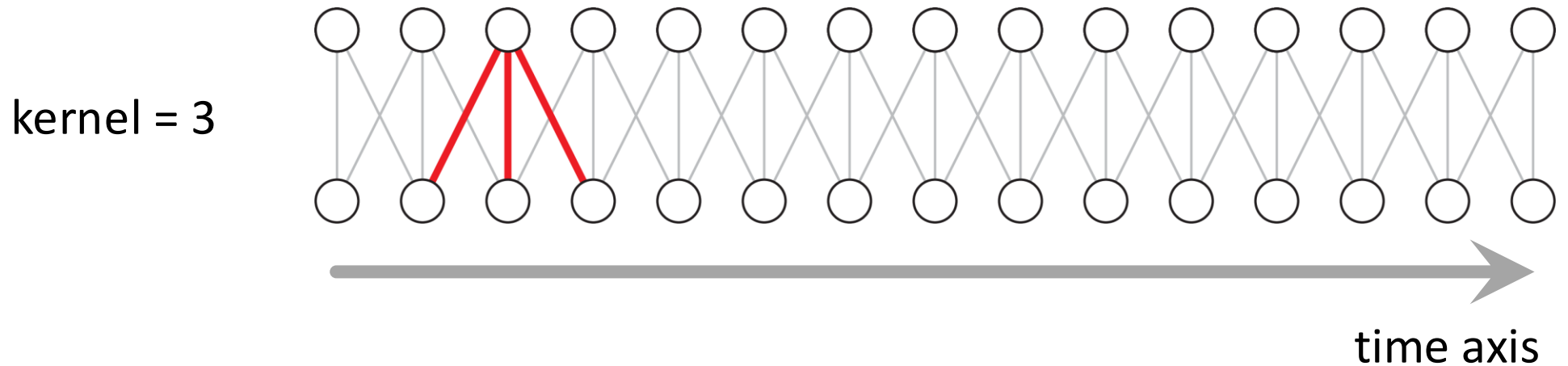
This lecture

CNNs for Sequences

1D ConvNet

Convolution along the time axis

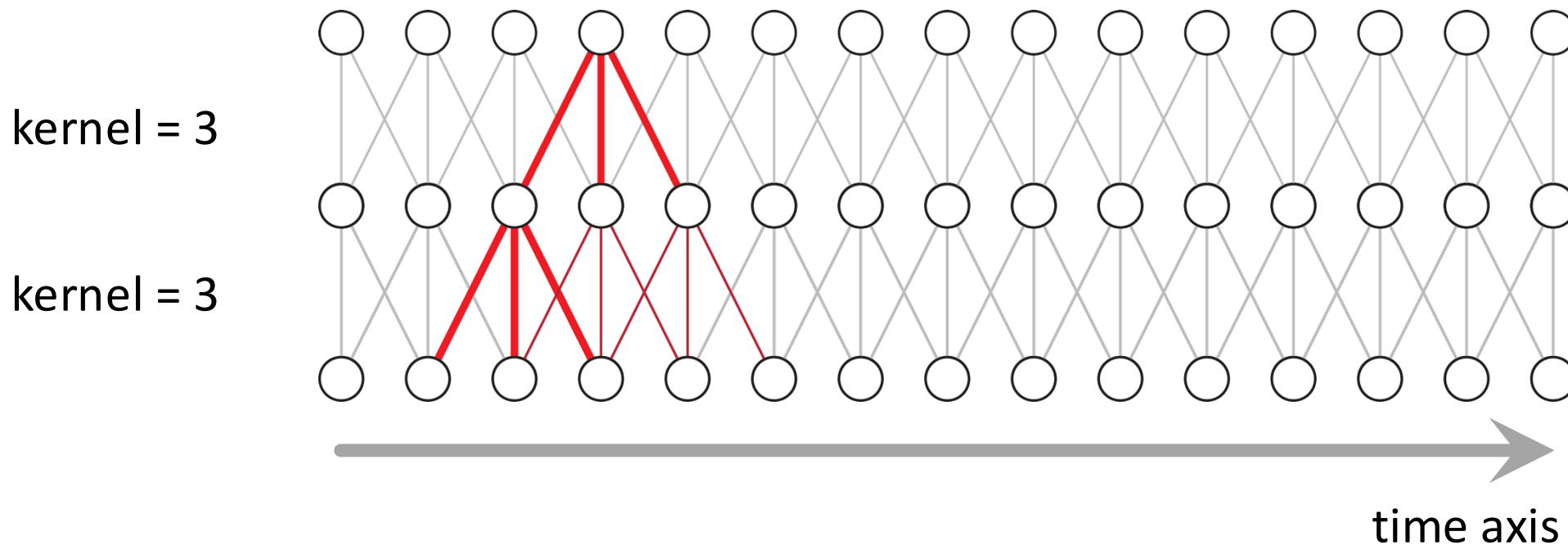
- sliding window: along time
- local connection: across time steps
- weight-sharing: across time steps



1D ConvNet

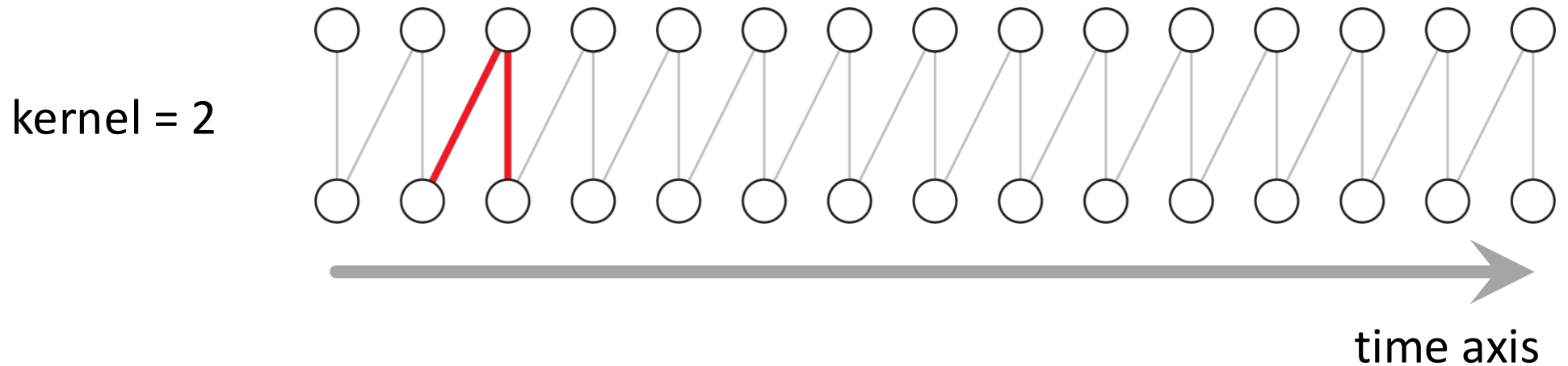
Convolution along the time axis

- sliding window: along time
- local connection: across time steps
- weight-sharing: across time steps



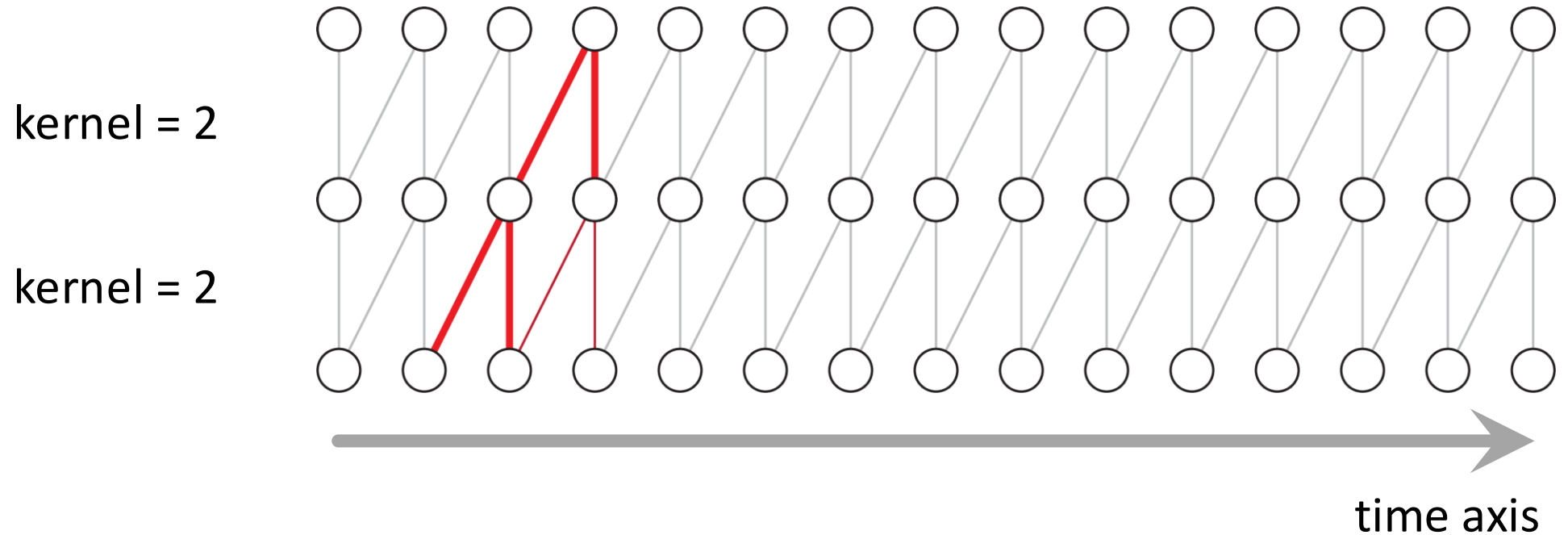
1D ConvNet: Causal convolution

- **Causal:** output at a time step does not depend on future time steps



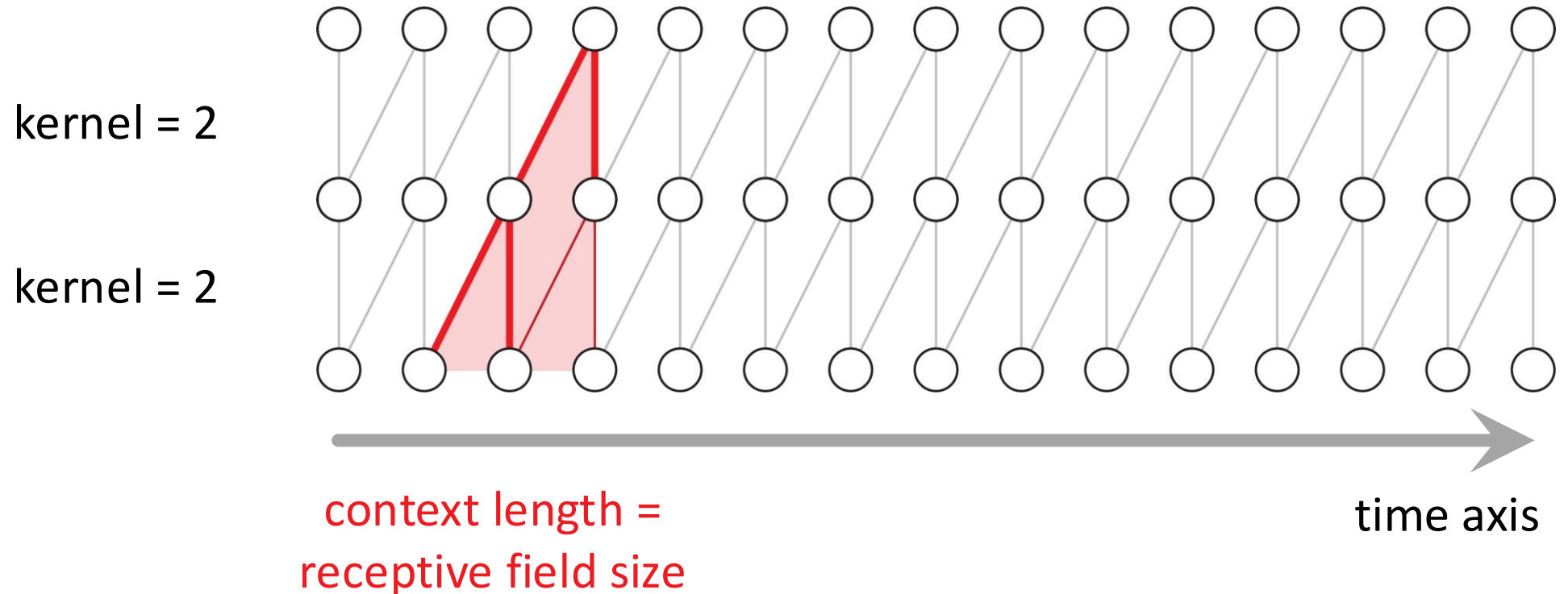
1D ConvNet: Causal convolution

- **Causal:** output at a time step does not depend on future time steps



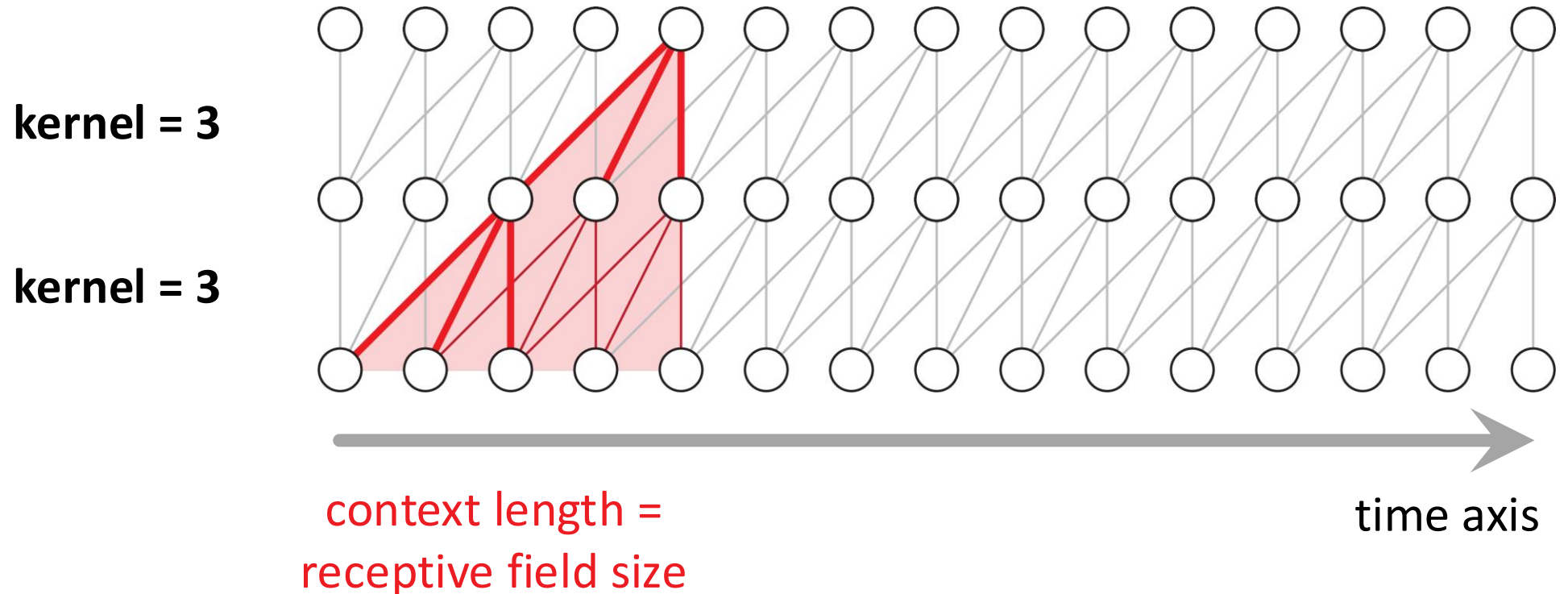
1D ConvNet: Causal convolution

- **Causal:** output at a time step does not depend on future time steps



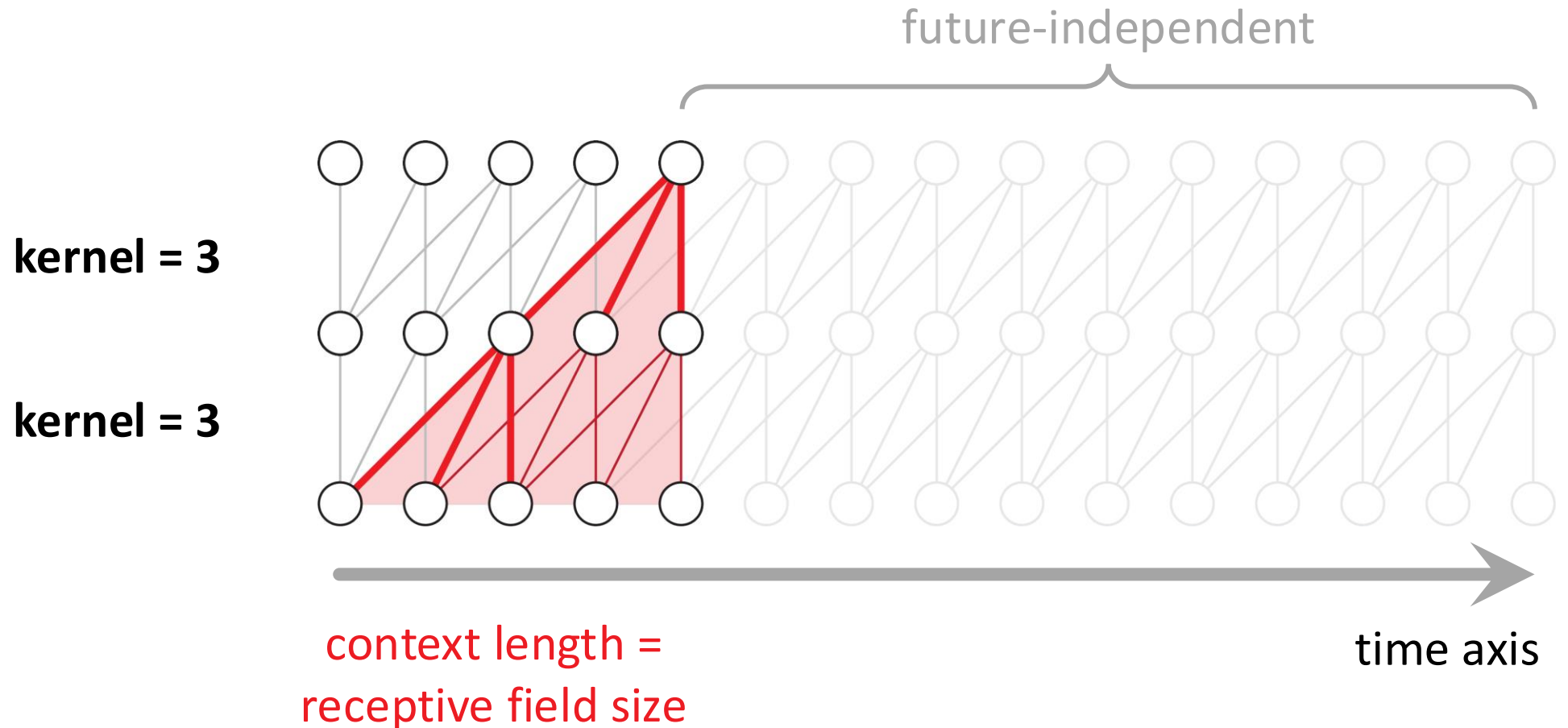
1D ConvNet: Causal convolution

- **Causal:** output at a time step does not depend on future time steps



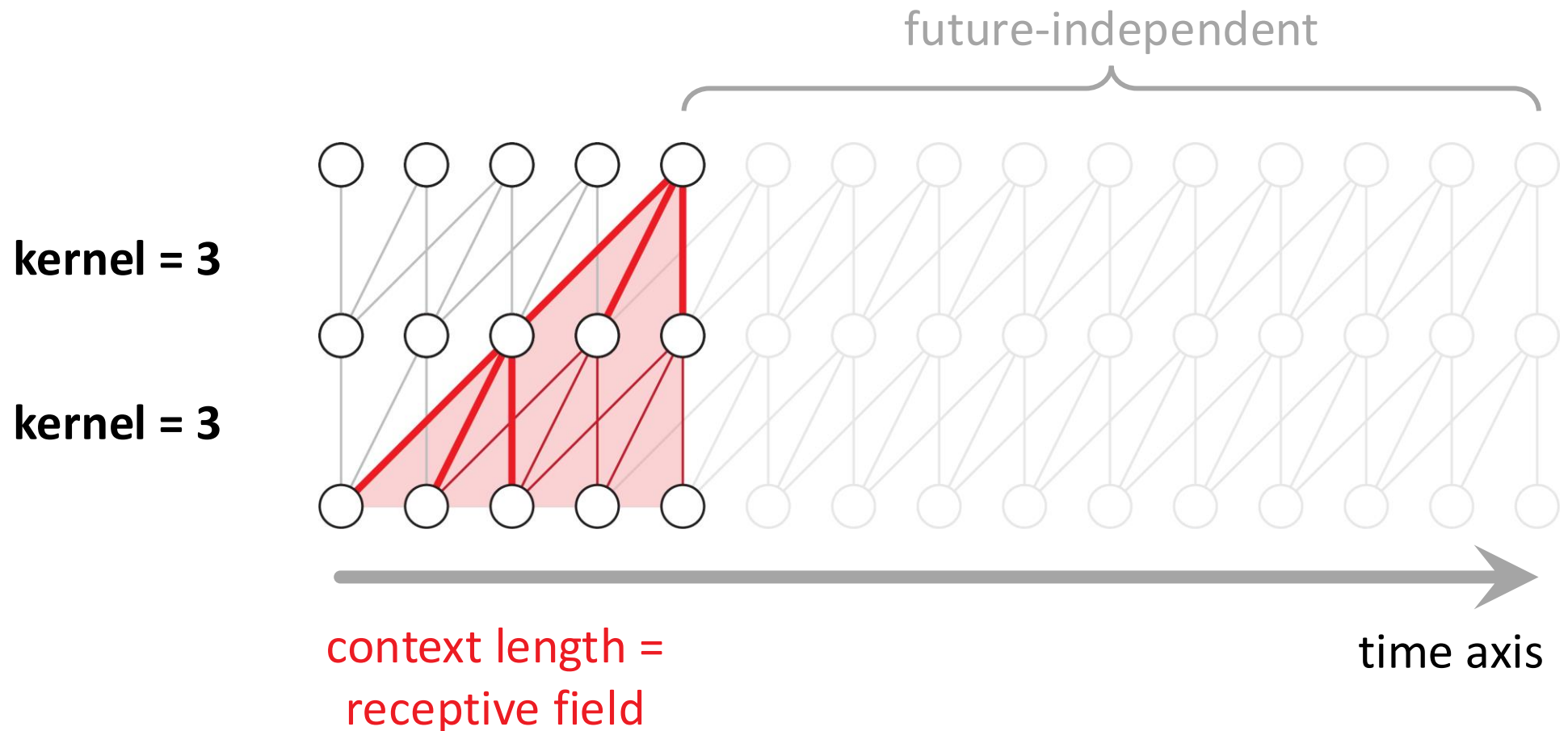
1D ConvNet: Causal convolution

- **Causal:** output at a time step does not depend on future time steps



1D ConvNet: Causal convolution

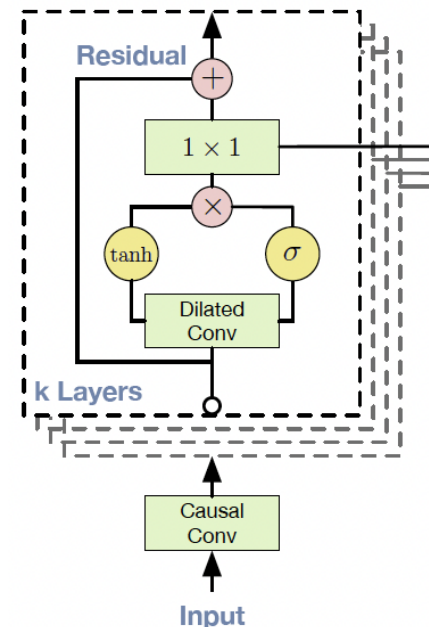
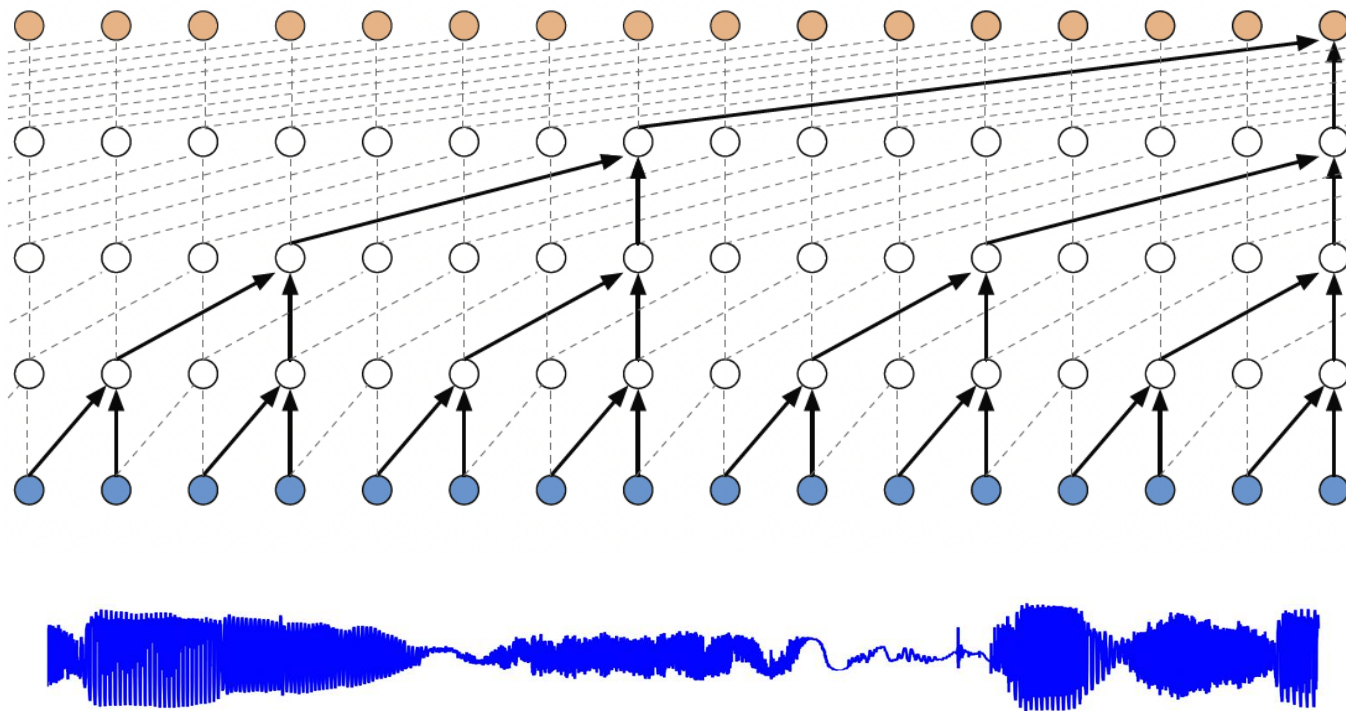
- **Causal:** output at a time step does not depend on future time steps
- Implementation: conv w/ **pad = kernel - 1**, only on “left” (past)



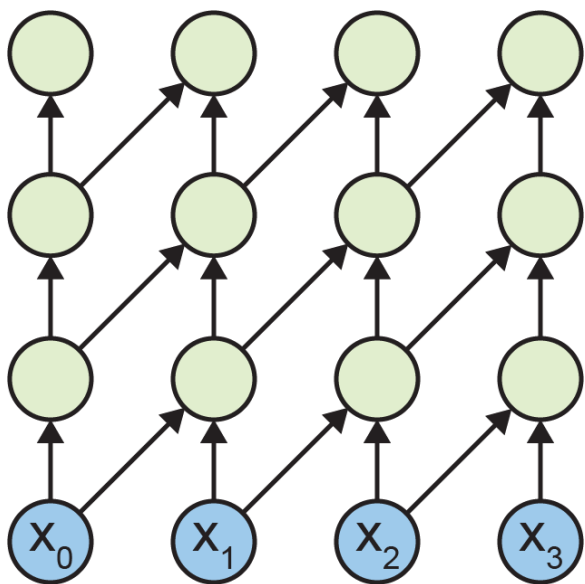
Example: DeepMind's WaveNet for Audio Generation

Causal ConvNet with long context

- going deep (with residual connections)
- “dilated” convolution (stride in kernel)

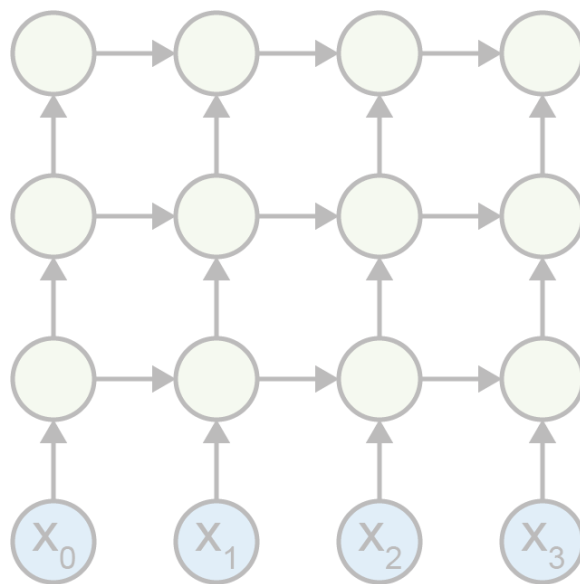


Paradigms of Sequence Modeling

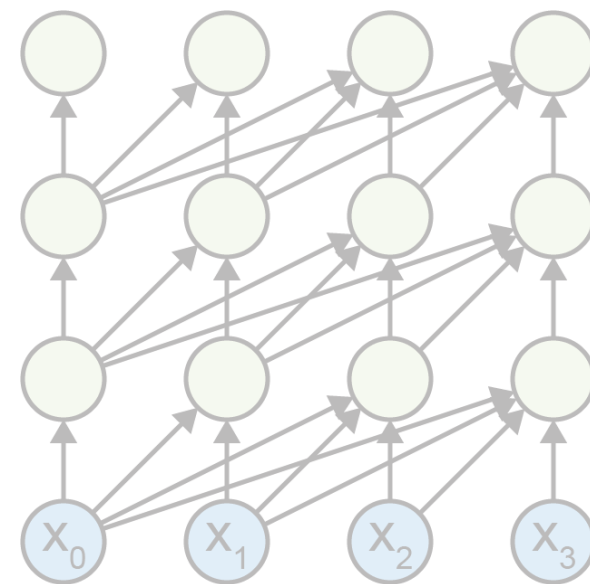


CNN

- parallelizable within layer
- context limited by depth



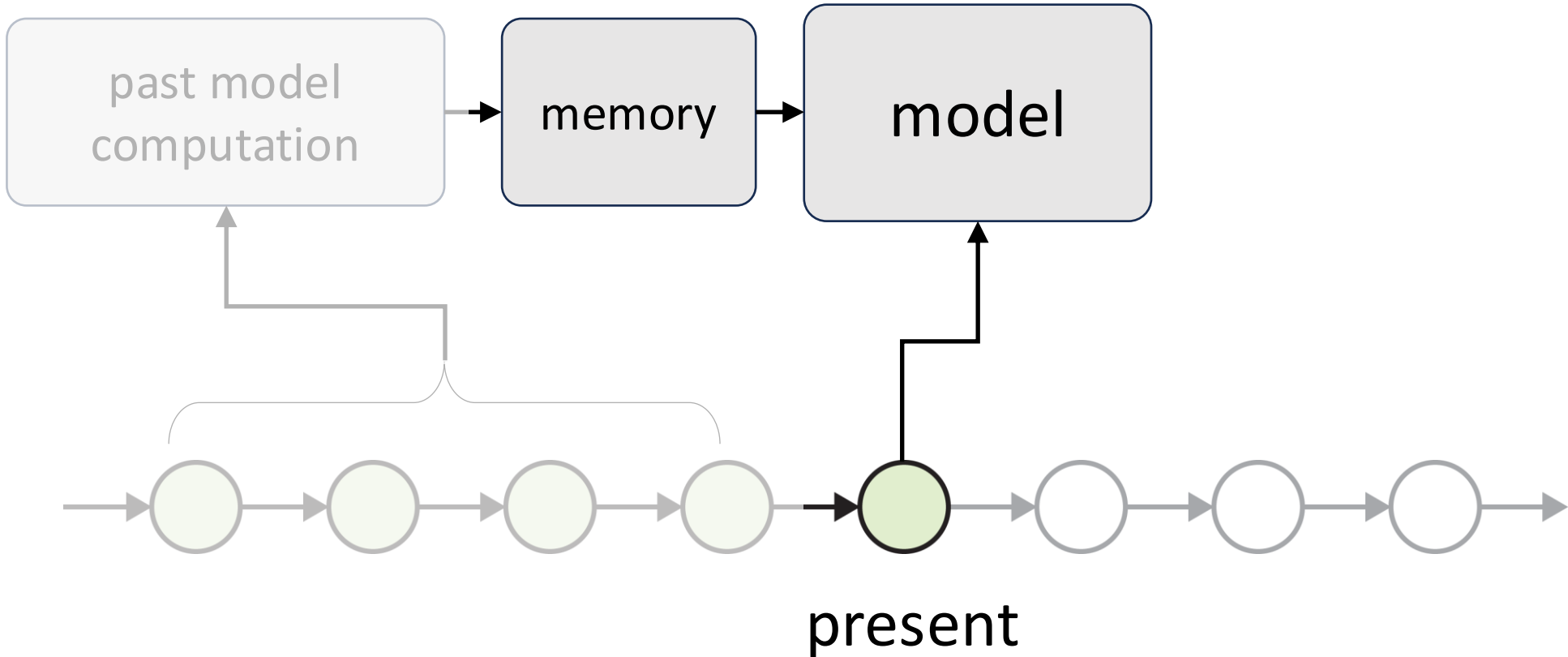
RNN



Attention

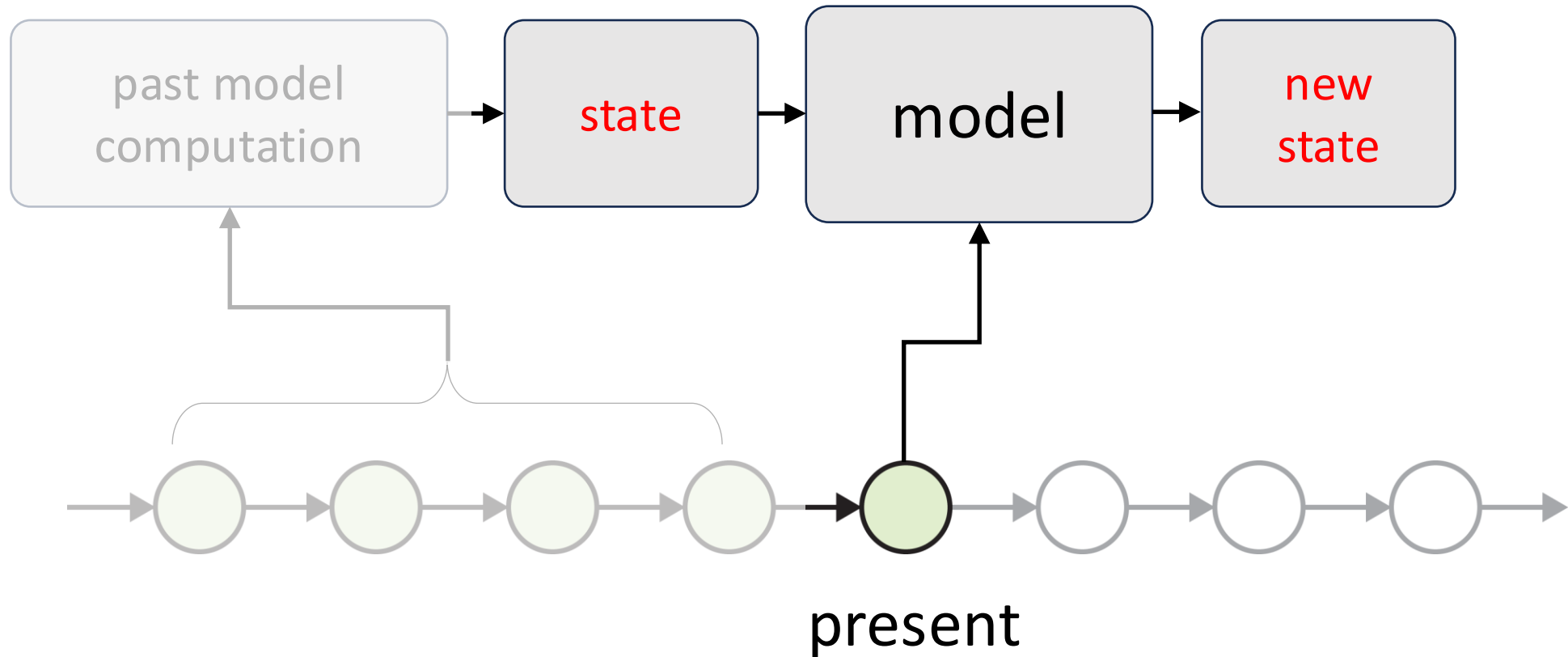
Recurrent Neural Networks (RNNs)

[Recap]: A General View of Sequence Modeling



- The model only sees present & memory

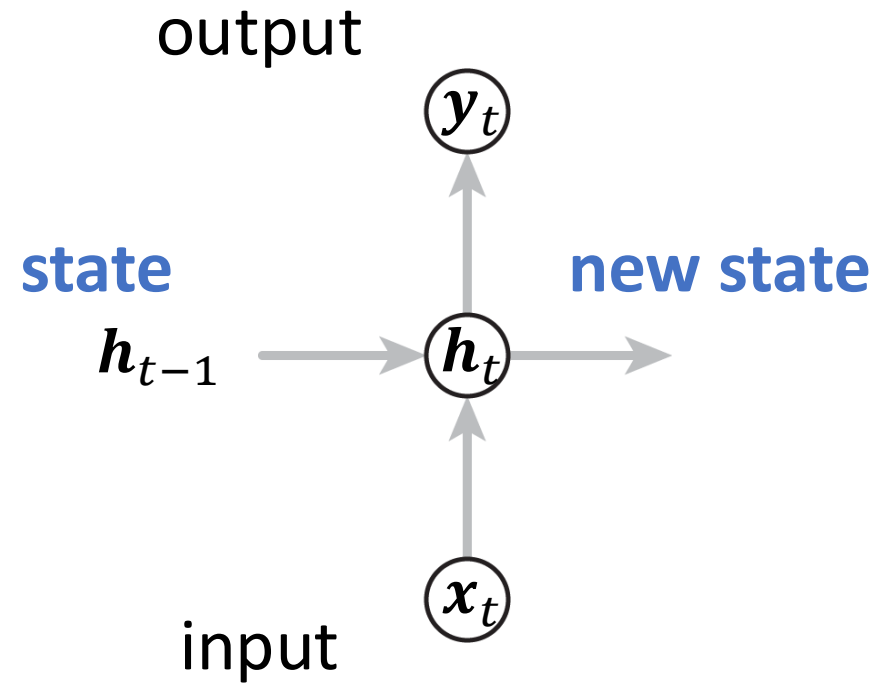
[Recap]: A General View of Sequence Modeling



- The model only sees present & memory

Recurrent NN: Hidden State

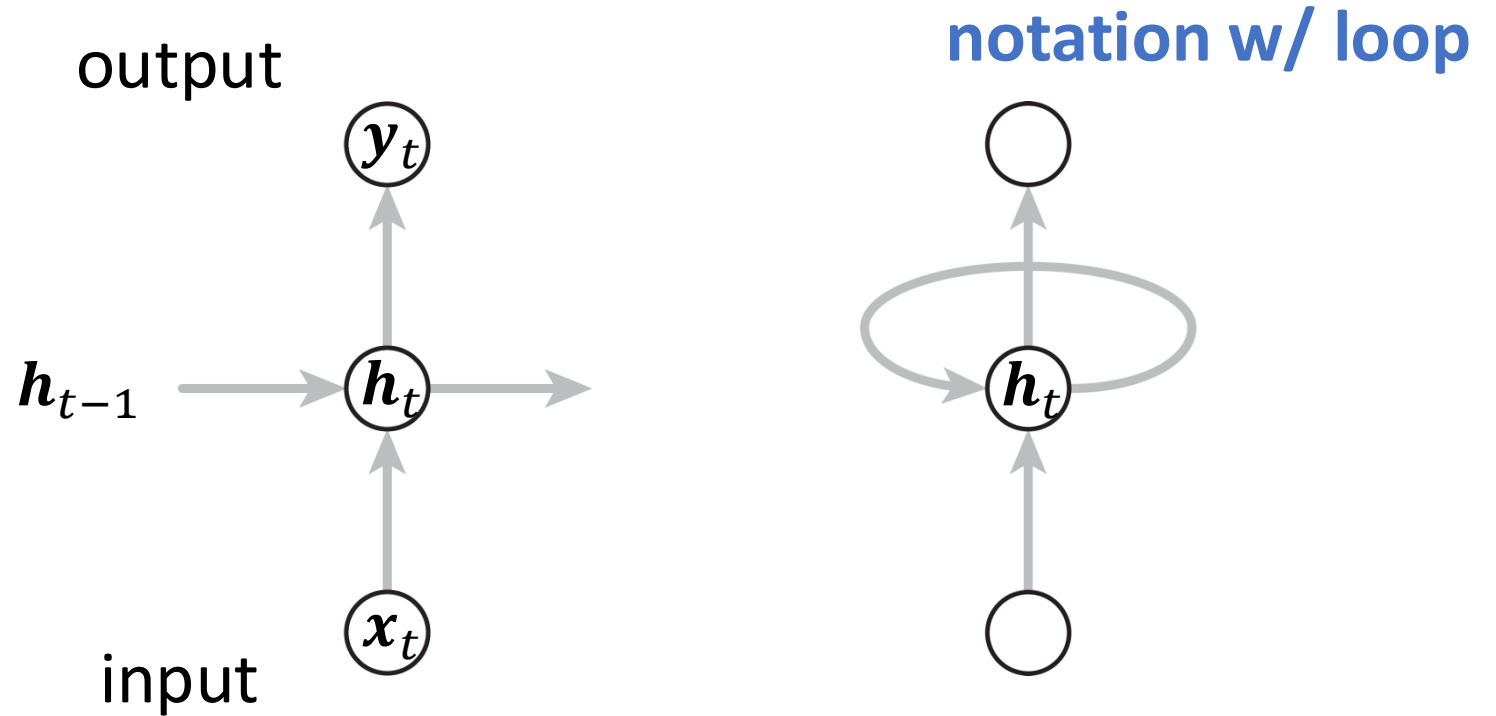
- Hidden state: a summary of the sequence observed so far



$$\begin{aligned} h_t &= f(h_{t-1}, x_t) & \bullet & \text{recurrent} \\ y_t &= g(h_t) & \bullet & \text{feedforward} \end{aligned}$$

Recurrent NN: Hidden State

- Hidden state: a summary of the sequence observed so far

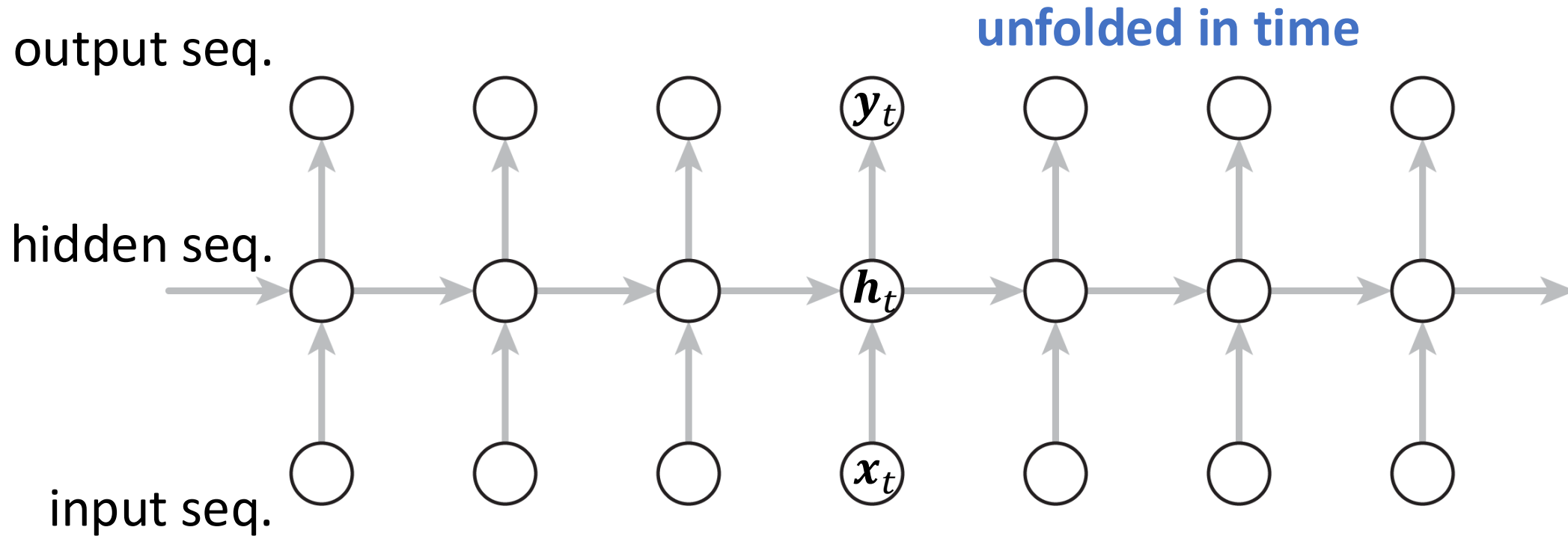


$$\begin{aligned} h_t &= f(h_{t-1}, x_t) \\ y_t &= g(h_t) \end{aligned}$$

- recurrent
- feedforward

Recurrent NN: Hidden State

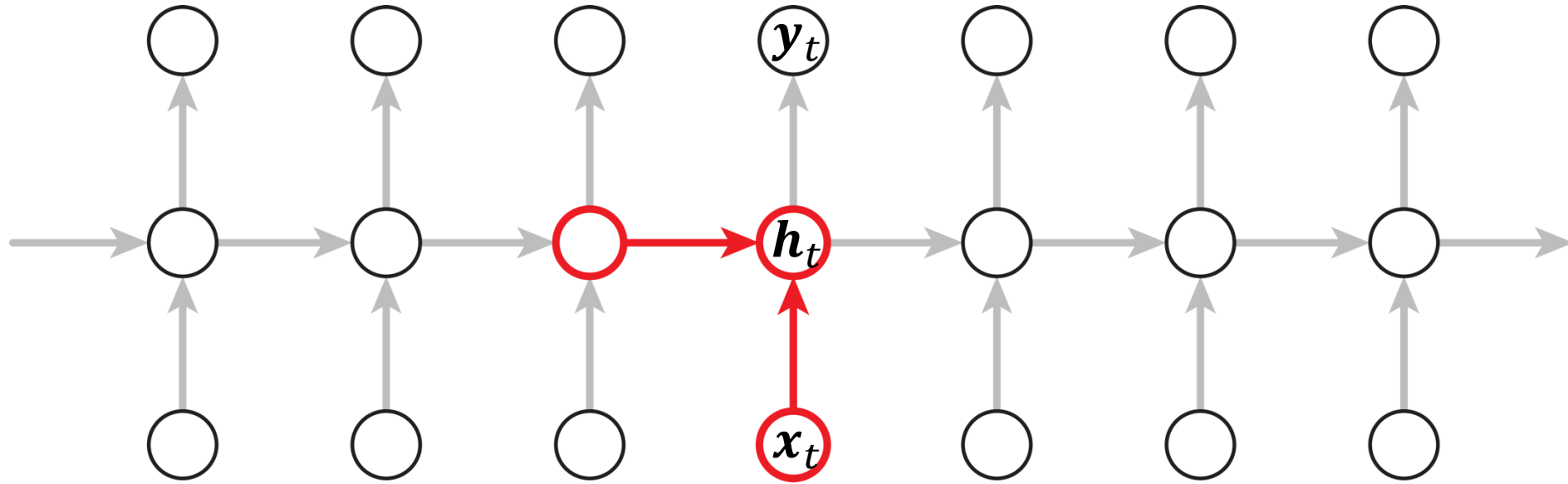
- Hidden state: a summary of the sequence observed so far



$$\begin{aligned} h_t &= f(h_{t-1}, x_t) & \bullet & \text{recurrent} \\ y_t &= g(h_t) & \bullet & \text{feedforward} \end{aligned}$$

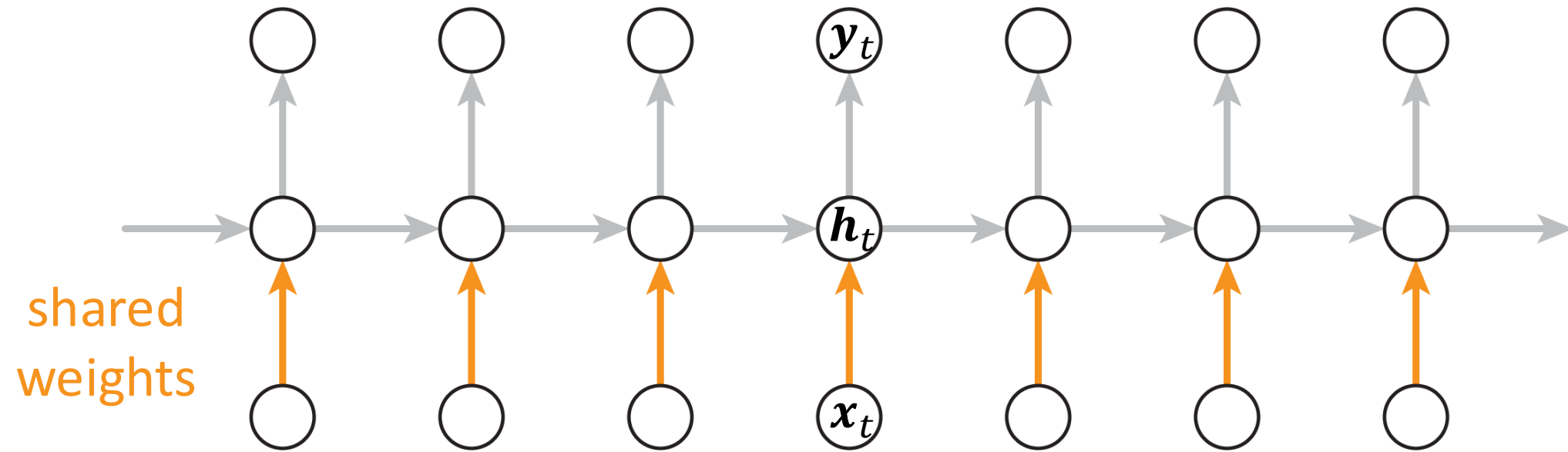
RNN: local connection + weight-sharing

- local connection: across time



RNN: local connection + weight-sharing

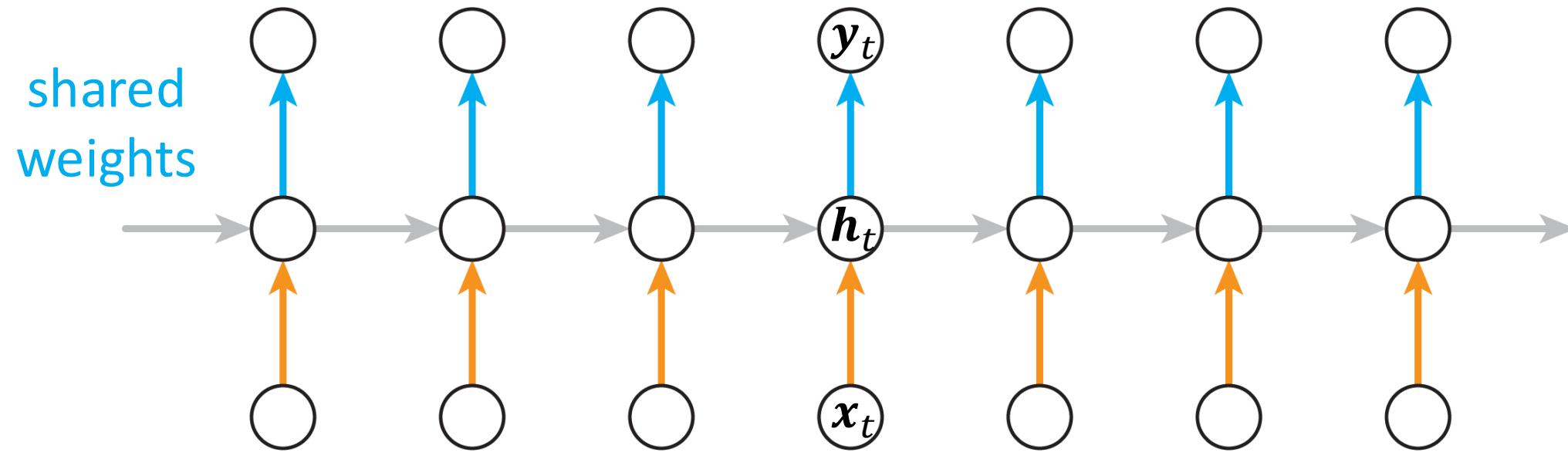
- weight-sharing: across time



similar to conv
with kernel size 1

RNN: local connection + weight-sharing

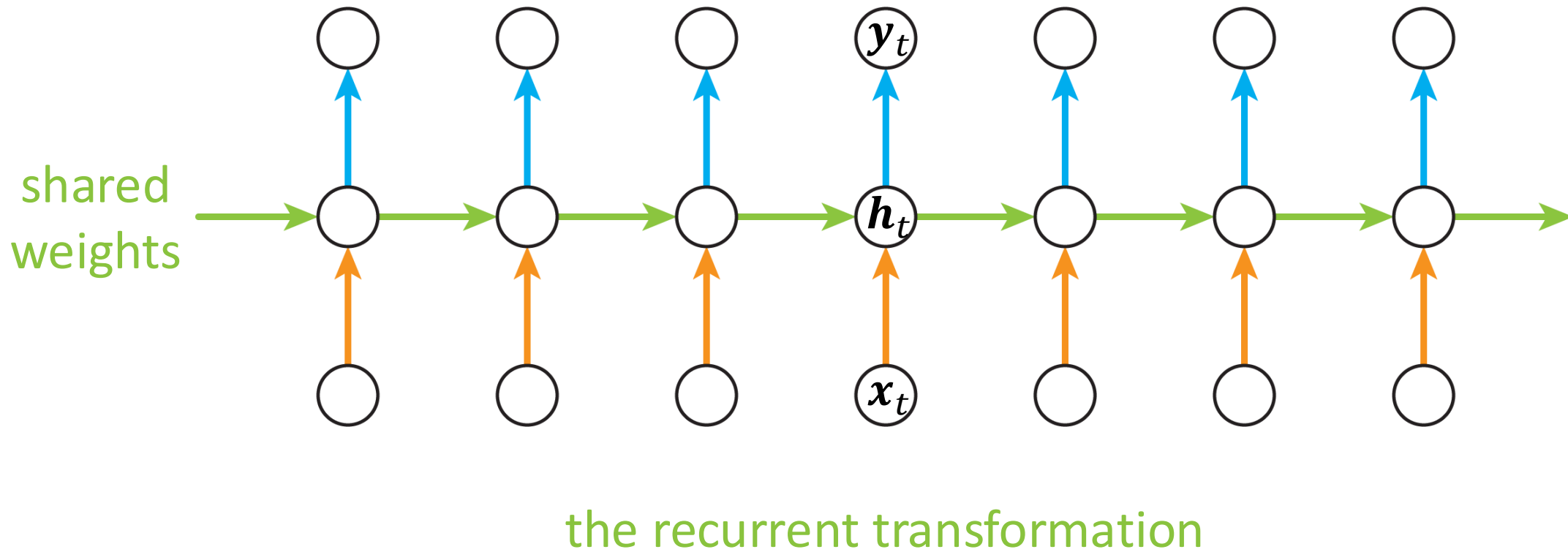
- weight-sharing: across time



similar to conv
with kernel size 1

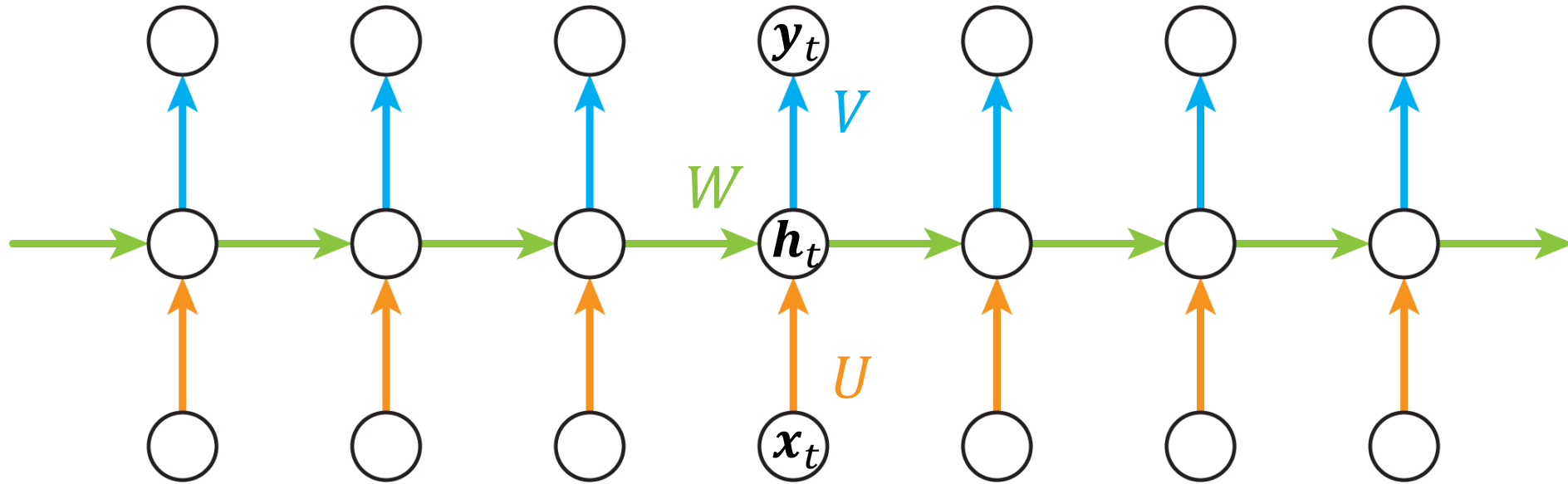
RNN: local connection + weight-sharing

- weight-sharing: across time



Example: “Vanilla” RNN

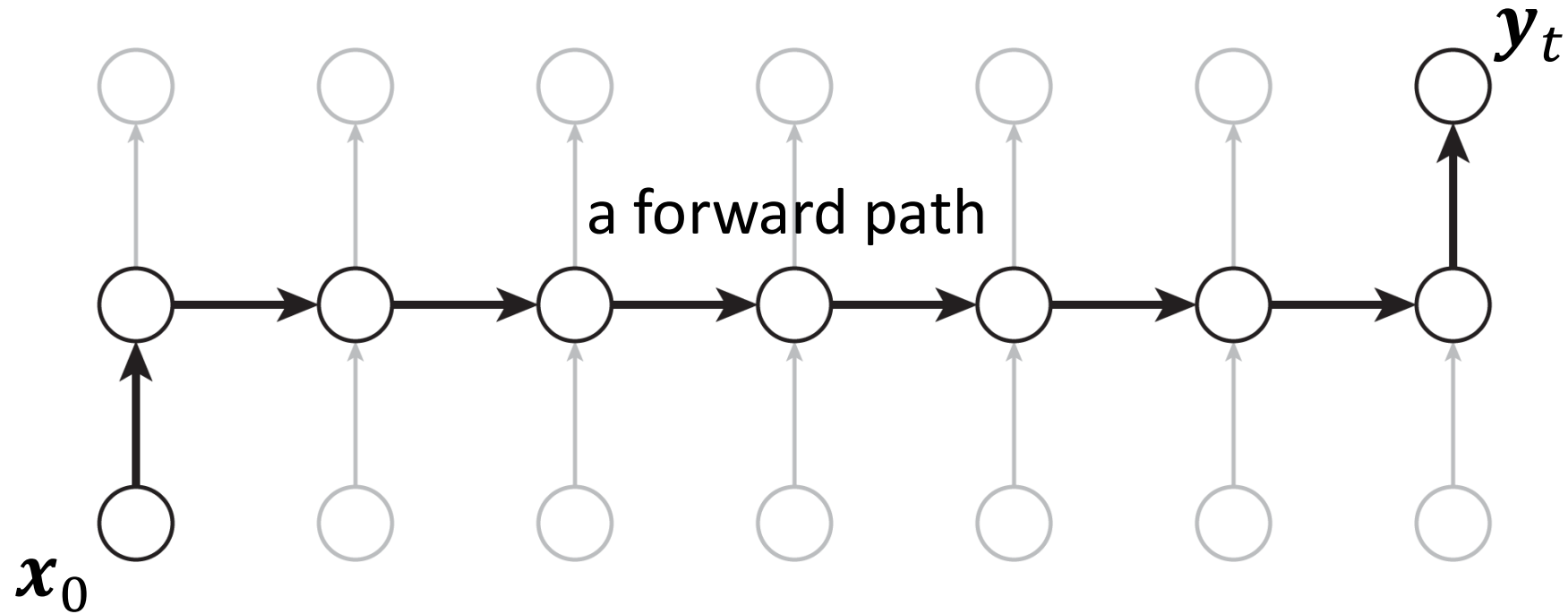
$$\begin{aligned} \mathbf{h}_t &= \tanh(\mathbf{W}\mathbf{h}_{t-1} + \mathbf{U}\mathbf{x}_t) \\ \mathbf{y}_t &= \mathbf{V}\mathbf{h}_t \end{aligned}$$



*bias omitted for simplicity

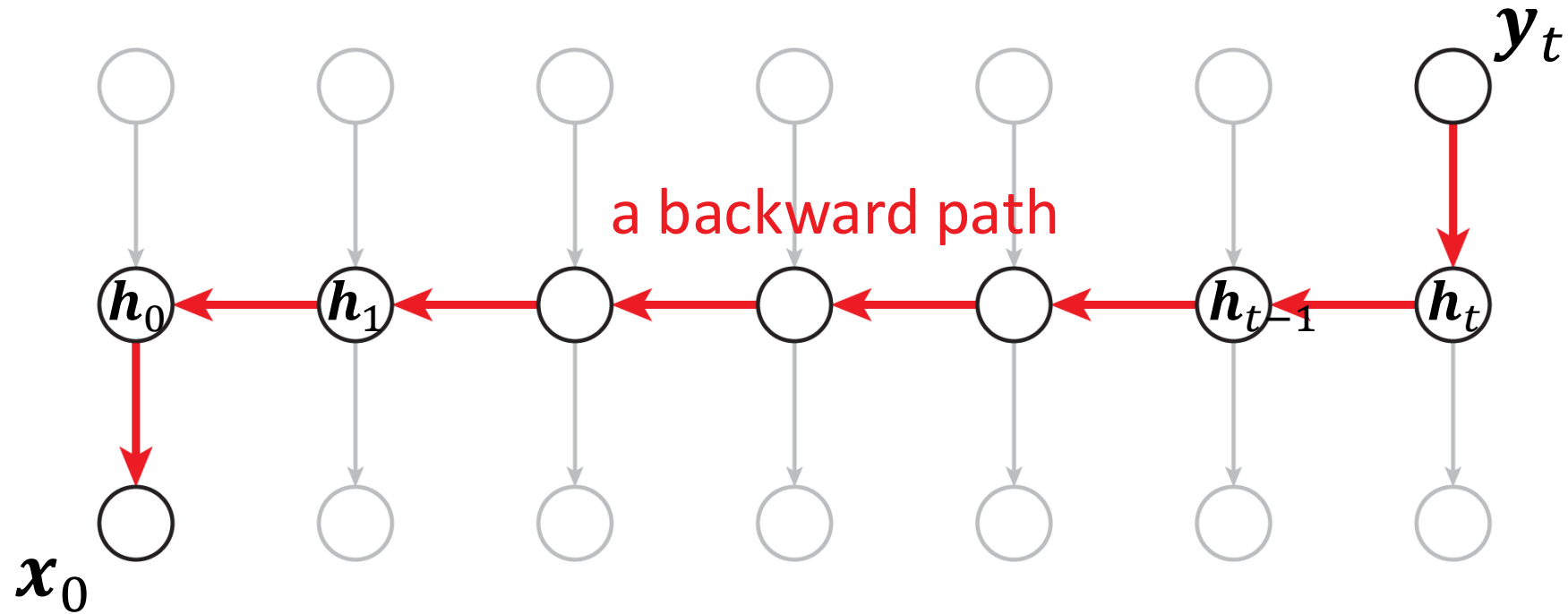
RNN: BackProp Through Time (BPTT)

- output at time t is like a t -layer deep net



RNN: BackProp Through Time (BPTT)

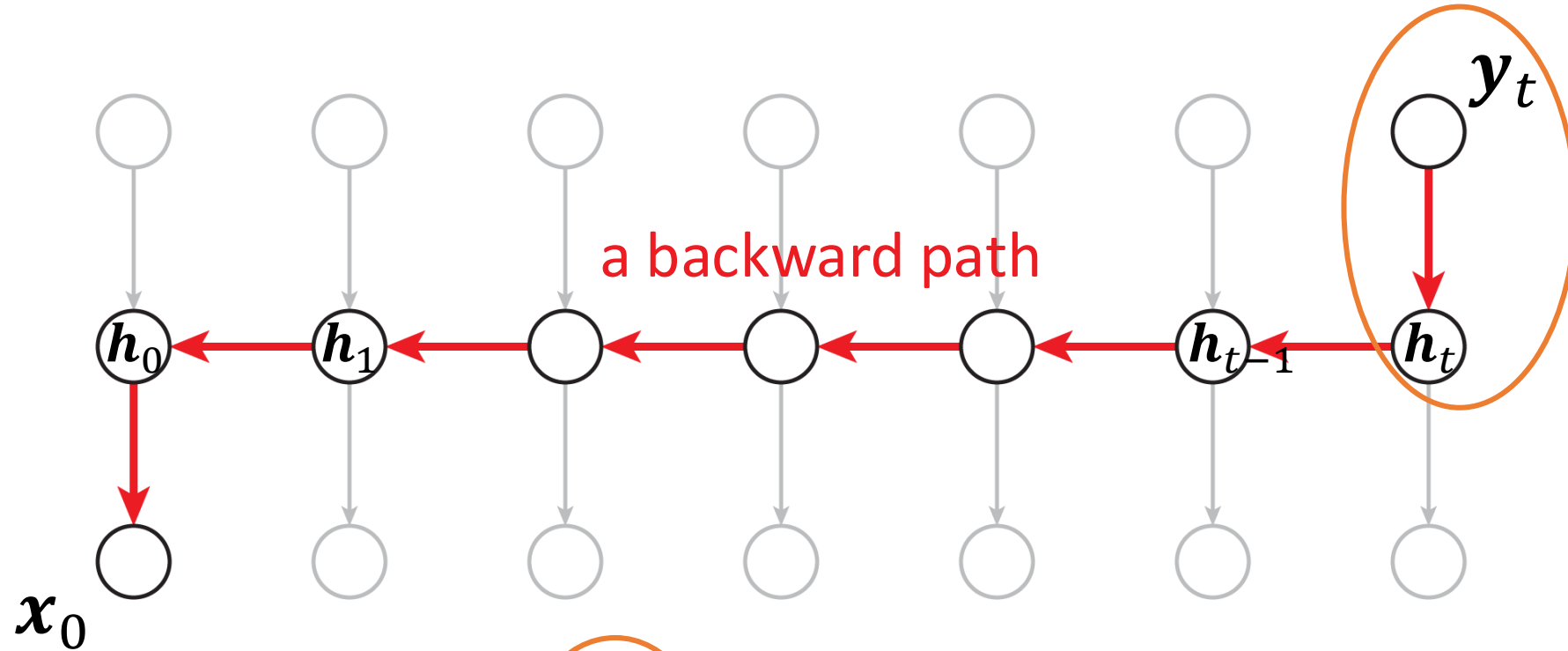
- output at time t is like a t -layer deep net



$$\frac{\partial \mathbf{y}_t}{\partial \mathbf{x}_0} = \frac{\partial \mathbf{y}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \cdots \frac{\partial \mathbf{h}_1}{\partial \mathbf{h}_0} \frac{\partial \mathbf{h}_0}{\partial \mathbf{x}_0}$$

RNN: BackProp Through Time (BPTT)

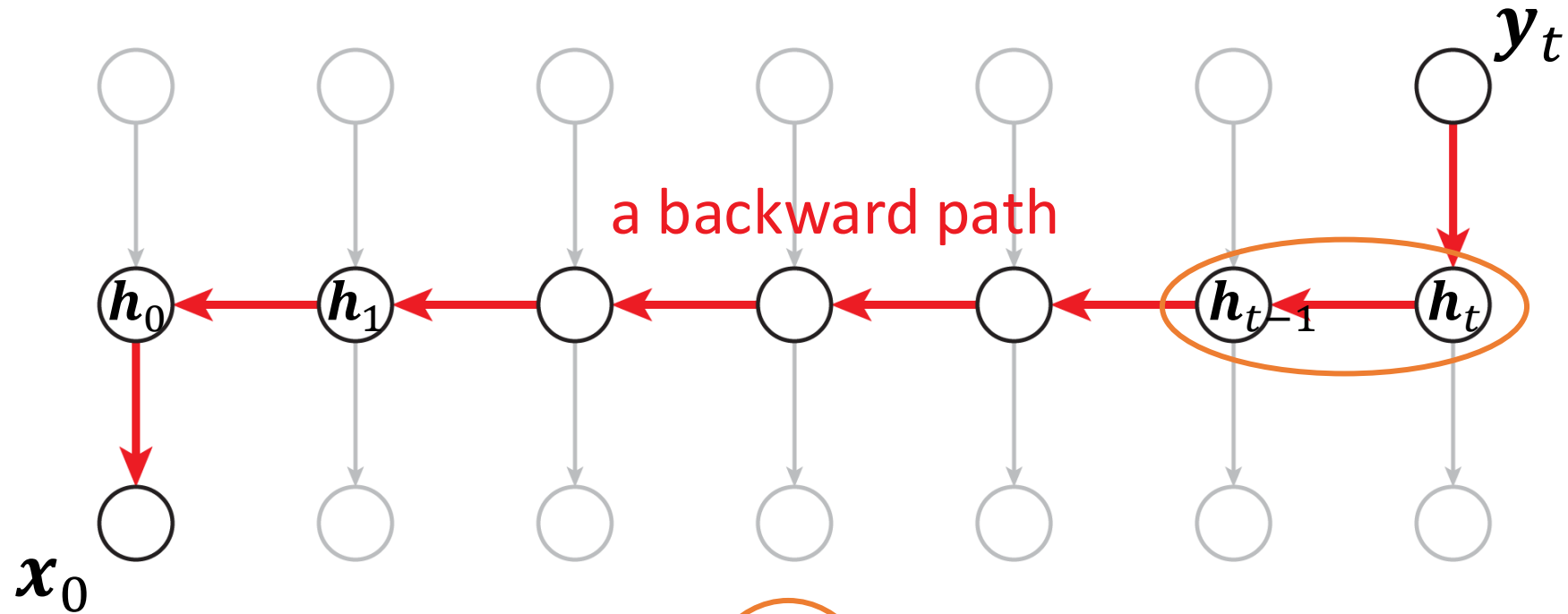
- output at time t is like a t -layer deep net



$$\frac{\partial y_t}{\partial x_0} = \frac{\partial y_t}{\partial h_t} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial h_0} \frac{\partial h_0}{\partial x_0}$$

RNN: BackProp Through Time (BPTT)

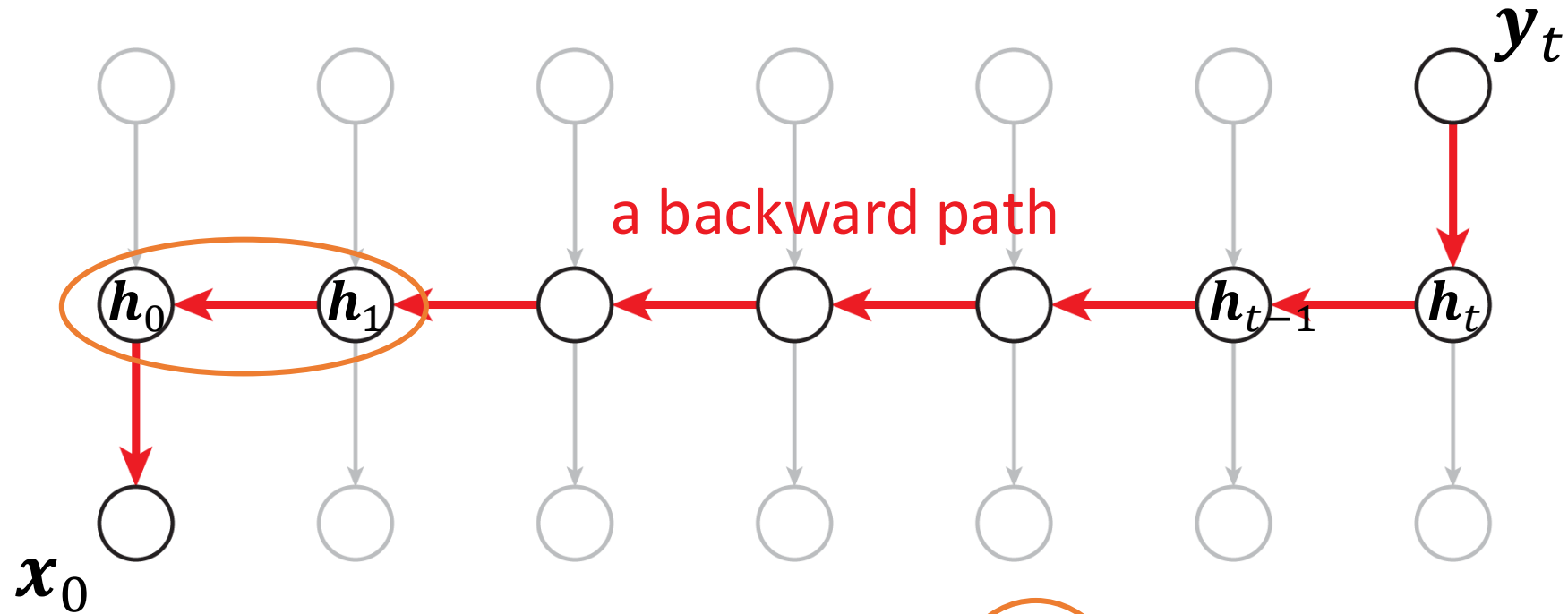
- output at time t is like a t -layer deep net



$$\frac{\partial \mathbf{y}_t}{\partial \mathbf{x}_0} = \frac{\partial \mathbf{y}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \cdots \frac{\partial \mathbf{h}_1}{\partial \mathbf{h}_0} \frac{\partial \mathbf{h}_0}{\partial \mathbf{x}_0}$$

RNN: BackProp Through Time (BPTT)

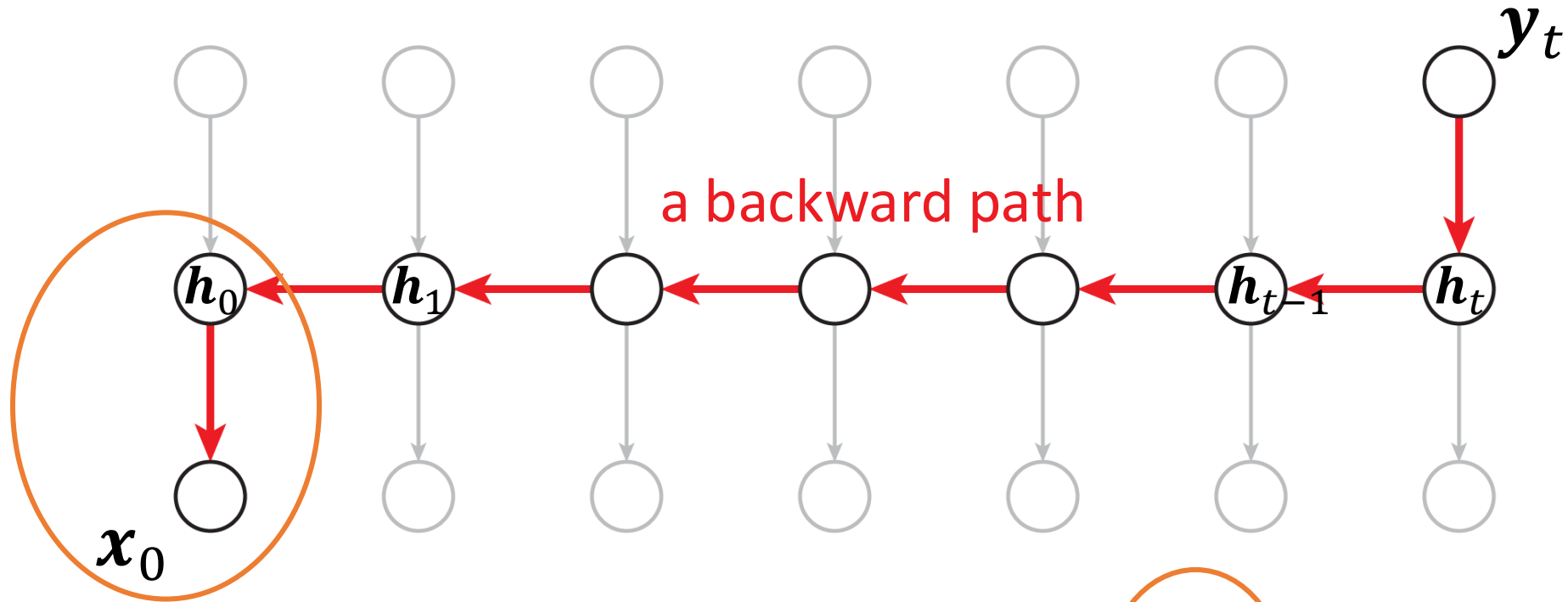
- output at time t is like a t -layer deep net



$$\frac{\partial \mathbf{y}_t}{\partial \mathbf{x}_0} = \frac{\partial \mathbf{y}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \cdots \frac{\partial \mathbf{h}_1}{\partial \mathbf{h}_0} \frac{\partial \mathbf{h}_0}{\partial \mathbf{x}_0}$$

RNN: BackProp Through Time (BPTT)

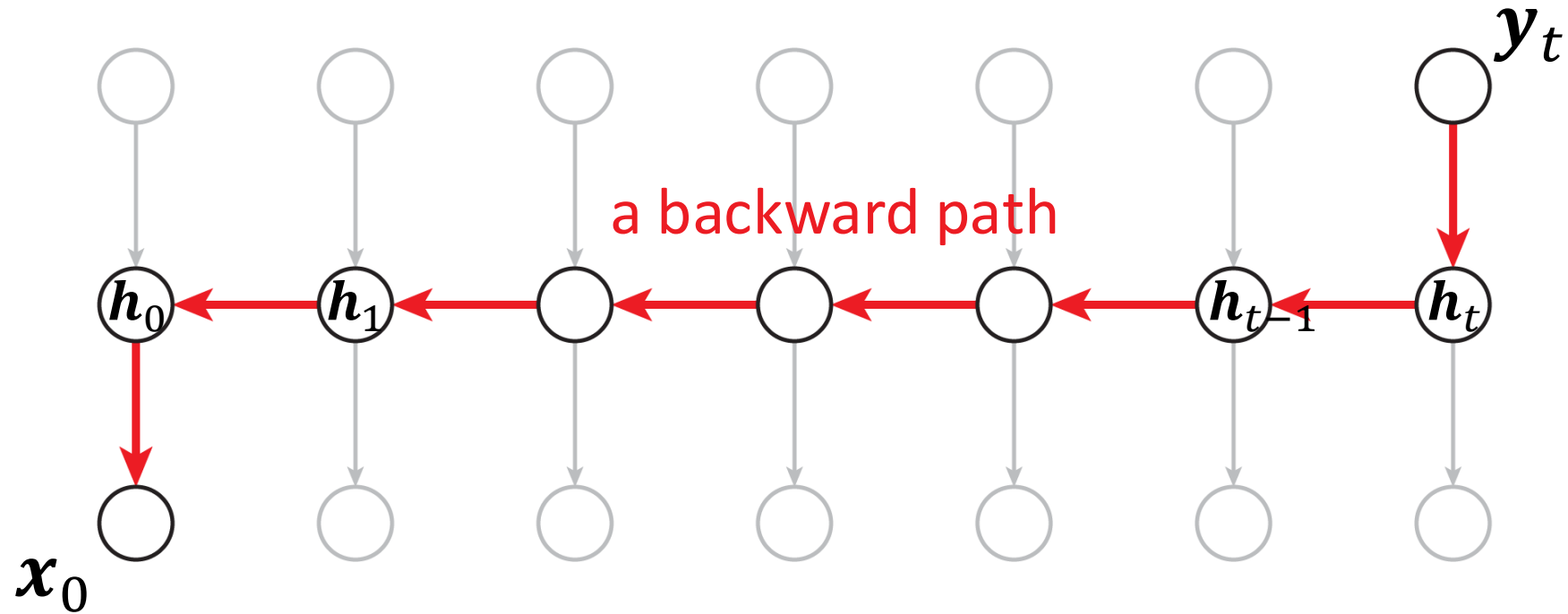
- output at time t is like a t -layer deep net



$$\frac{\partial y_t}{\partial x_0} = \frac{\partial y_t}{\partial h_t} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial h_0} \frac{\partial h_0}{\partial x_0}$$

RNN: BackProp Through Time (BPTT)

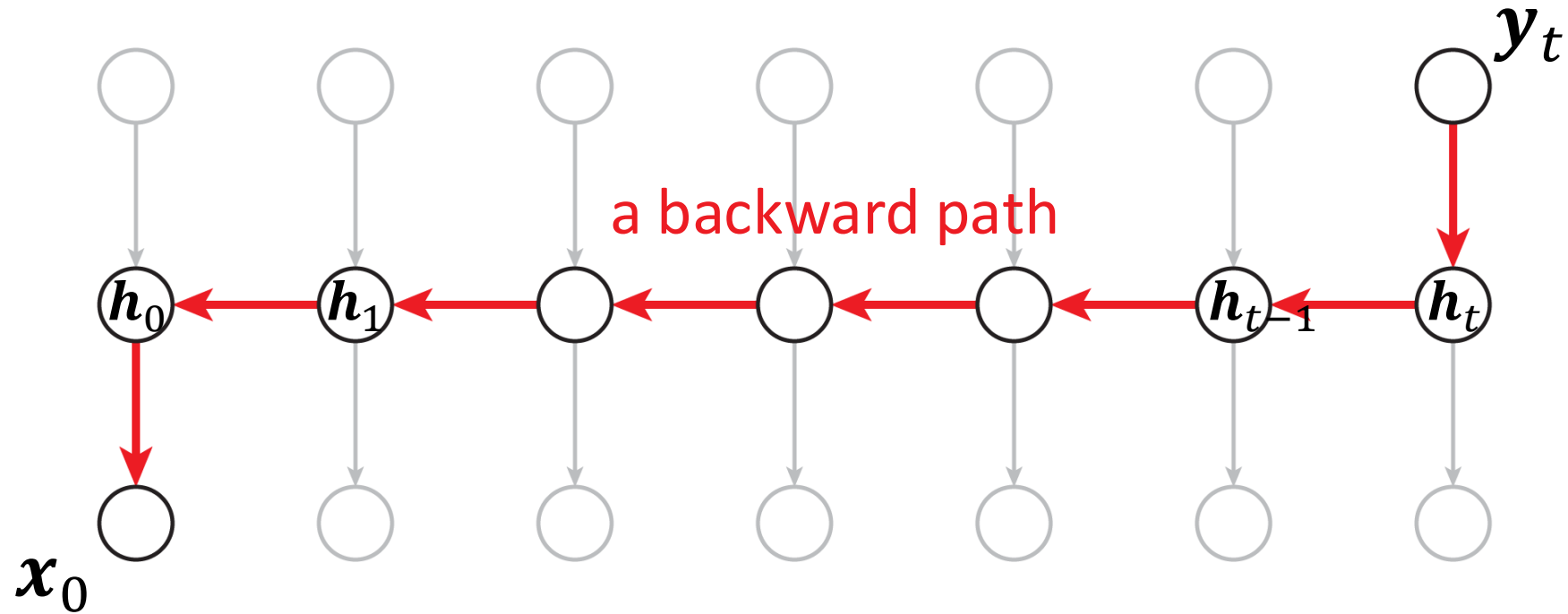
Example (linear vanilla RNN): $\mathbf{h}_t = W\mathbf{h}_{t-1} + U\mathbf{x}_t$



$$\frac{\partial \mathbf{y}_t}{\partial \mathbf{x}_0} = \frac{\partial \mathbf{y}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \cdots \frac{\partial \mathbf{h}_1}{\partial \mathbf{h}_0} \frac{\partial \mathbf{h}_0}{\partial \mathbf{x}_0}$$

RNN: BackProp Through Time (BPTT)

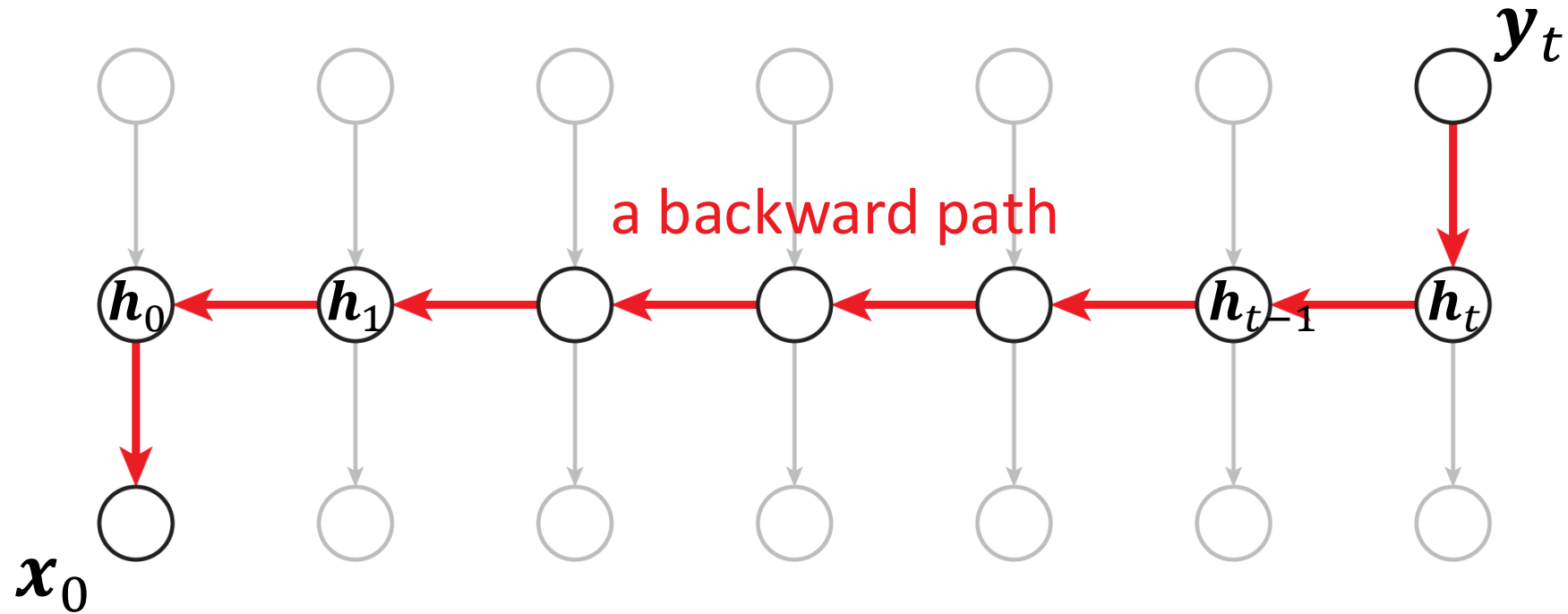
Example (linear vanilla RNN): $\mathbf{h}_t = W\mathbf{h}_{t-1} + U\mathbf{x}_t$



$$\frac{\partial \mathbf{y}_t}{\partial \mathbf{x}_0} = \frac{\partial \mathbf{y}_t}{\partial \mathbf{h}_t} \mathbf{W} \cdots \frac{\partial \mathbf{h}_1}{\partial \mathbf{h}_0} \frac{\partial \mathbf{h}_0}{\partial \mathbf{x}_0}$$

RNN: BackProp Through Time (BPTT)

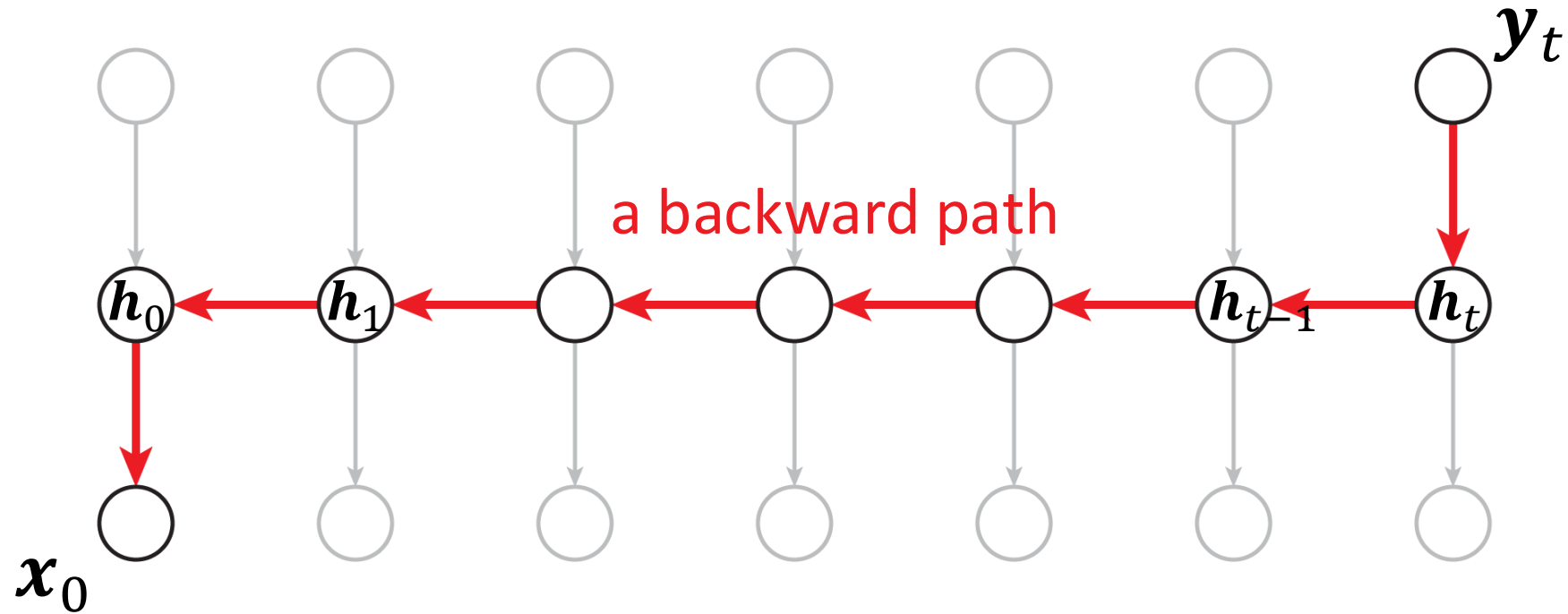
Example (linear vanilla RNN): $\mathbf{h}_t = W\mathbf{h}_{t-1} + U\mathbf{x}_t$



$$\frac{\partial \mathbf{y}_t}{\partial \mathbf{x}_0} = \frac{\partial \mathbf{y}_t}{\partial \mathbf{h}_t} \underbrace{W \dots W}_{t \times} \frac{\partial \mathbf{h}_0}{\partial \mathbf{x}_0}$$

RNN: BackProp Through Time (BPTT)

Example (linear vanilla RNN): $\mathbf{h}_t = W\mathbf{h}_{t-1} + U\mathbf{x}_t$



$$\frac{\partial \mathbf{y}_t}{\partial \mathbf{x}_0} = \frac{\partial \mathbf{y}_t}{\partial \mathbf{h}_t} W^t \frac{\partial \mathbf{h}_0}{\partial \mathbf{x}_0}$$

Case Study: Long Short-Term Memory (LSTM)

- LSTM is a form of RNN

- Memory

- summary of past

- Short-term memory

- only access nearby states (“Markov”)

- Long {Short-term memory}

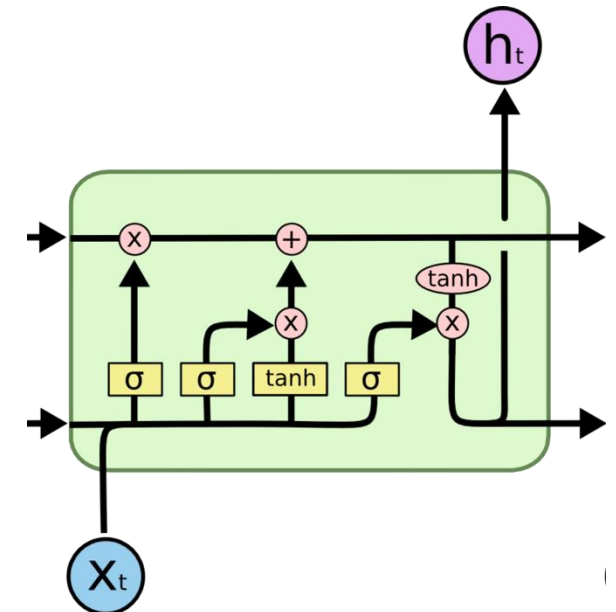
- attempt to retain information across a long time

LONG SHORT-TERM MEMORY

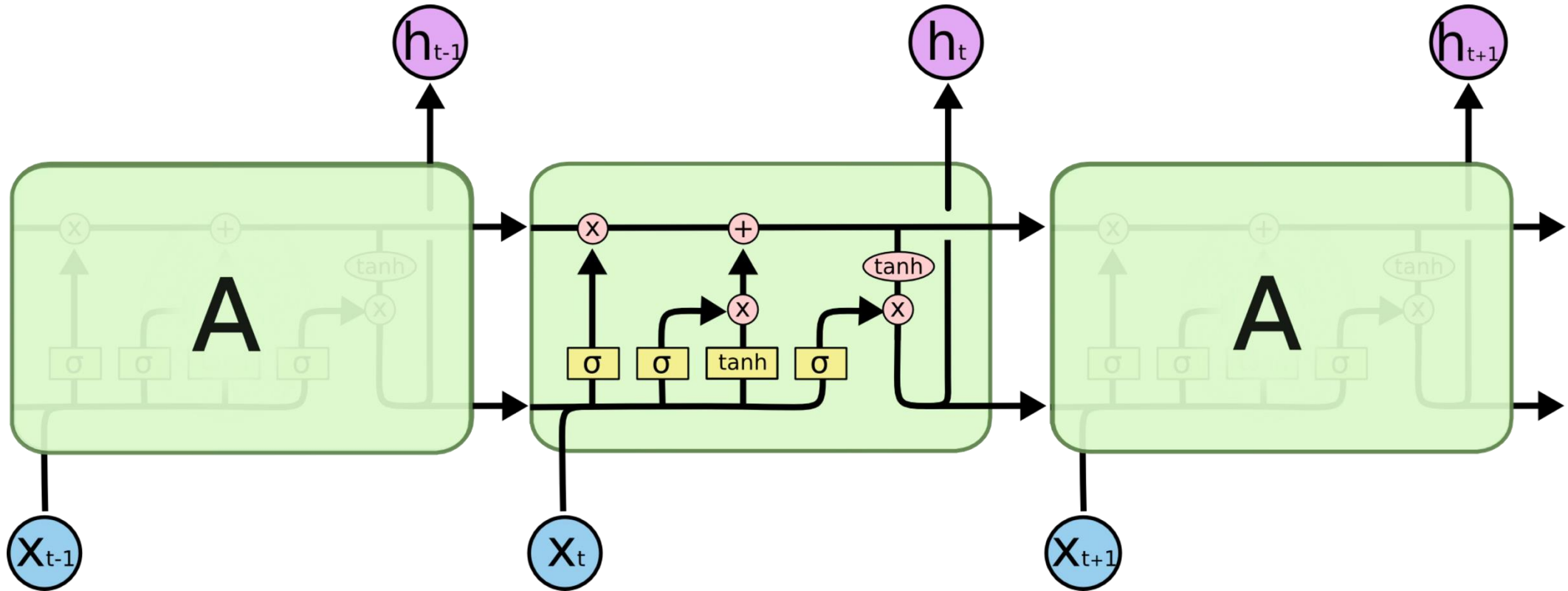
NEURAL COMPUTATION 9(8):1735–1780, 1997

Sepp Hochreiter
Fakultät für Informatik
Technische Universität München
80290 München, Germany
hochreit@informatik.tu-muenchen.de
<http://www7.informatik.tu-muenchen.de/~hochreit>

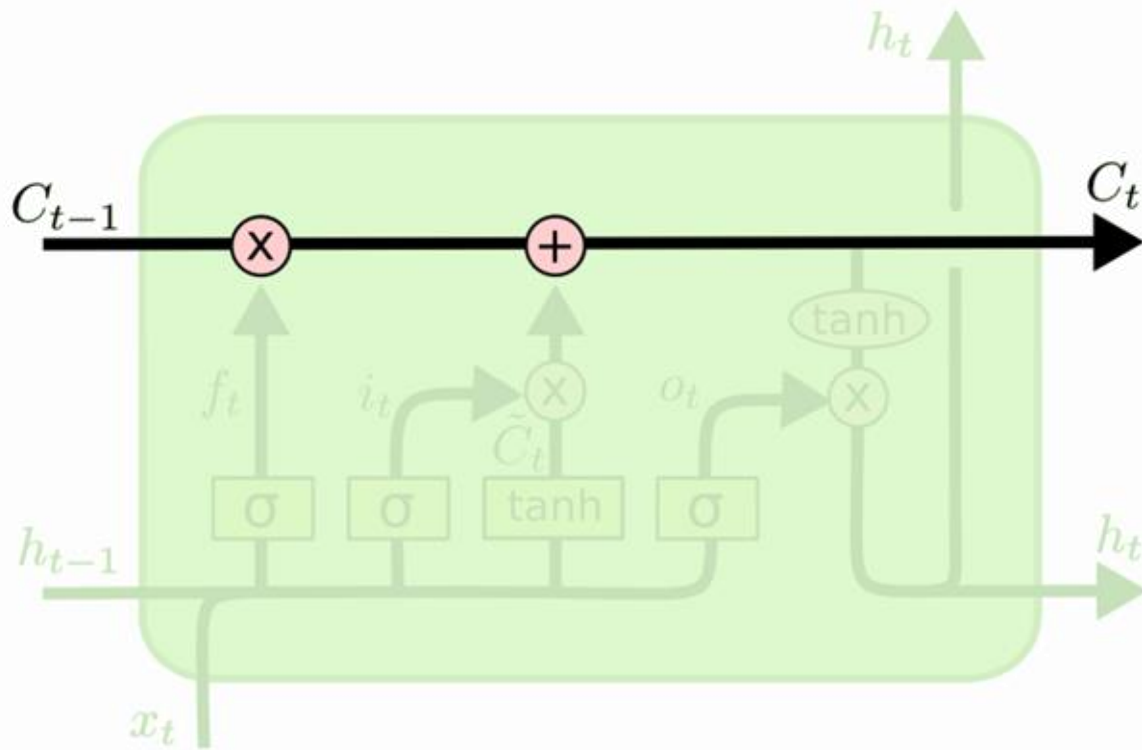
Jürgen Schmidhuber
IDSIA
Corso Elvezia 36
6900 Lugano, Switzerland
juergen@idsia.ch
<http://www.idsia.ch/~juergen>



Case Study: Long Short-Term Memory (LSTM)

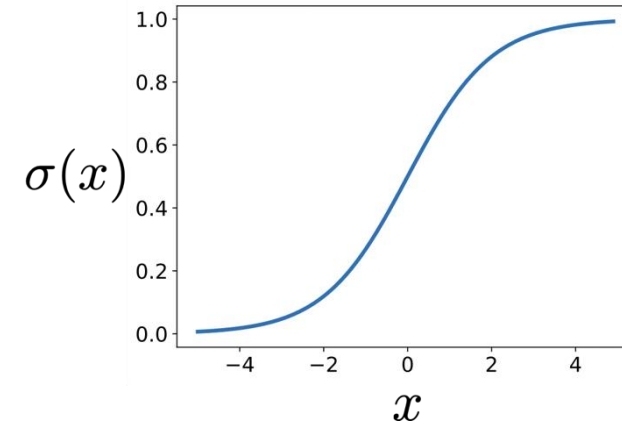
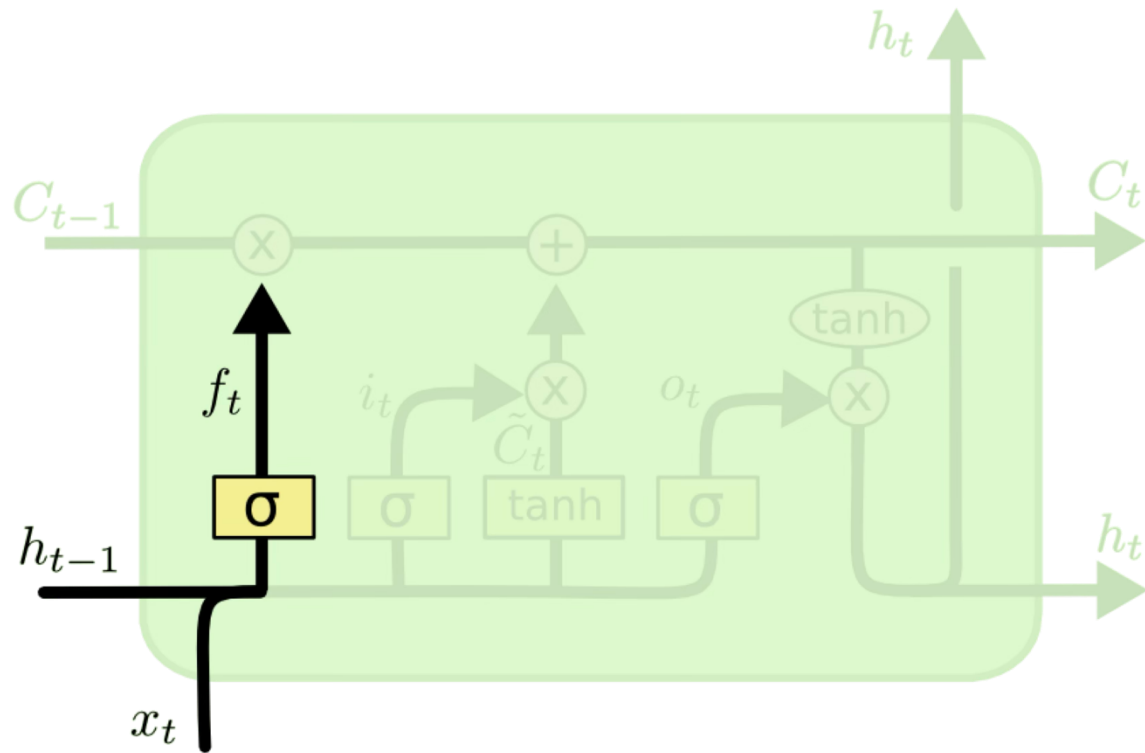


Case Study: Long Short-Term Memory (LSTM)



- C_t : Cell state
- h_t : hidden state

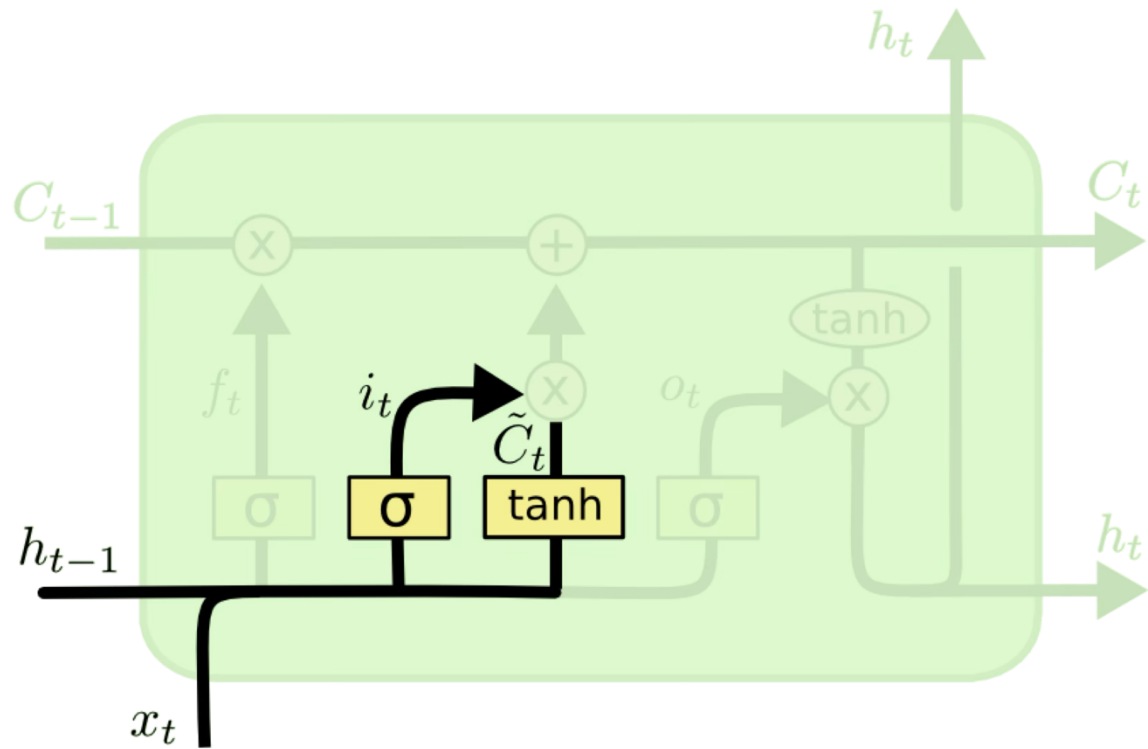
Case Study: Long Short-Term Memory (LSTM)



$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

- **Forget gate:** Controls how much of each cell-state component to retain.

Case Study: Long Short-Term Memory (LSTM)

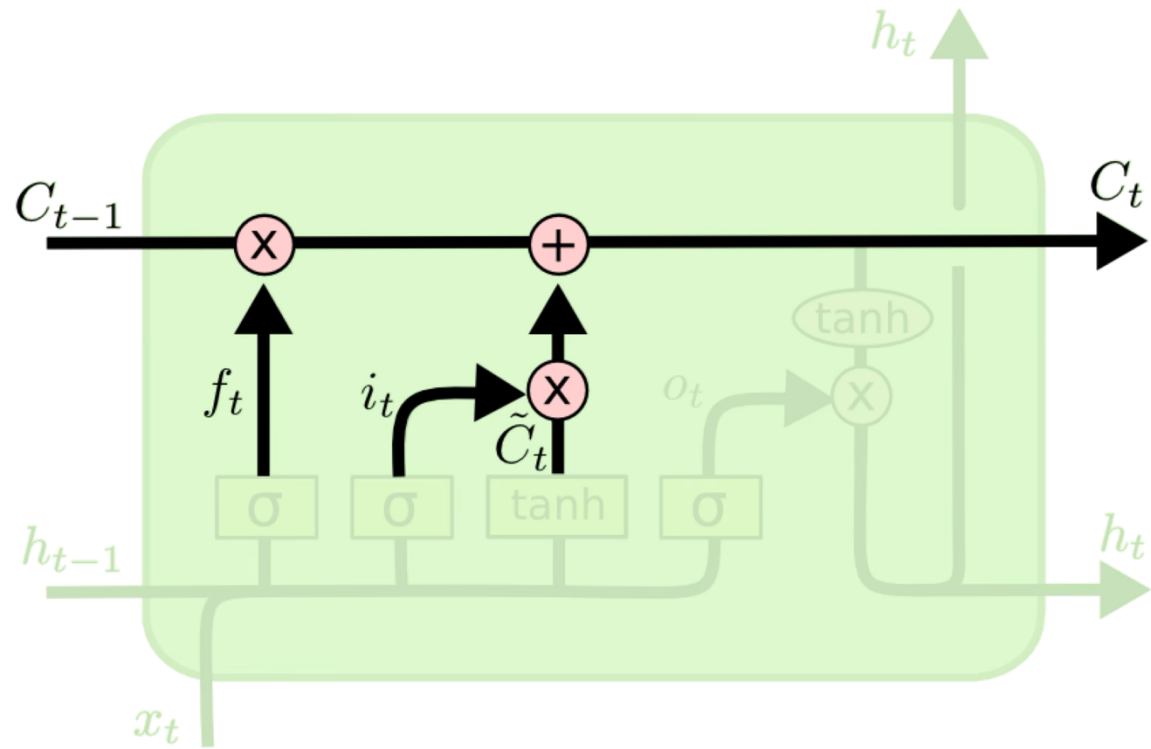


$$i_t = \sigma (W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

- **Input gate:** how much to write
- **Candidate memory:** what to write

Case Study: Long Short-Term Memory (LSTM)

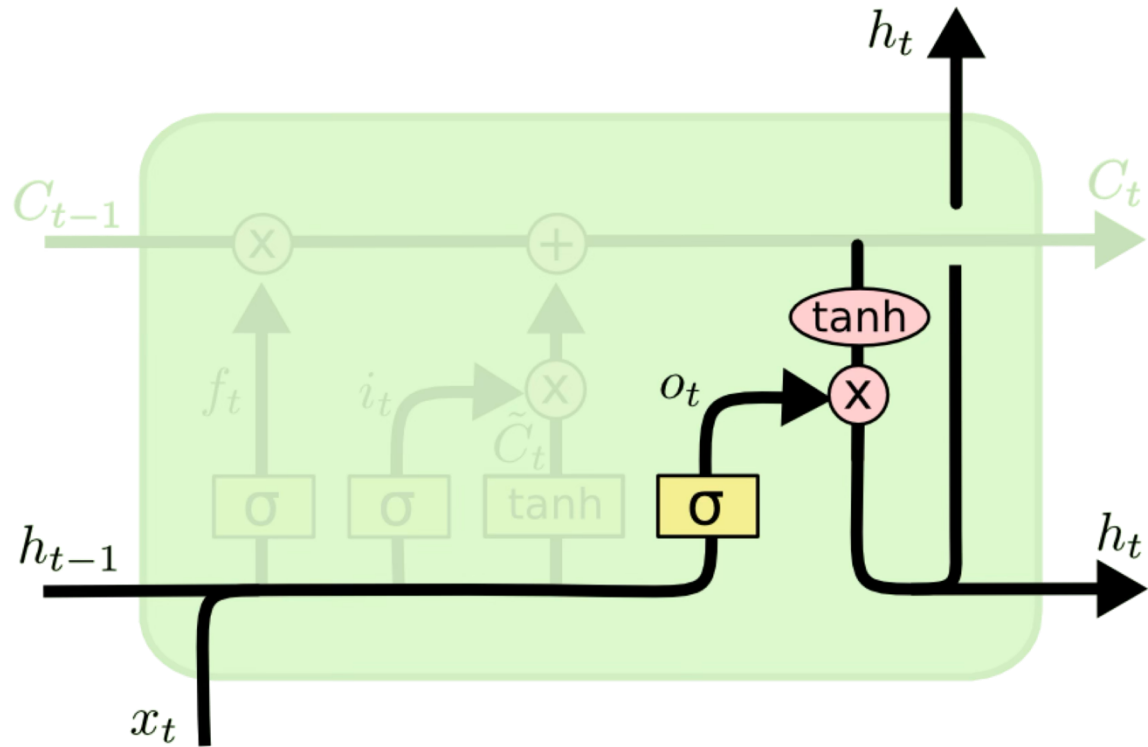


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Update the cell state selectively:

- forget old information
- write new information

Case Study: Long Short-Term Memory (LSTM)

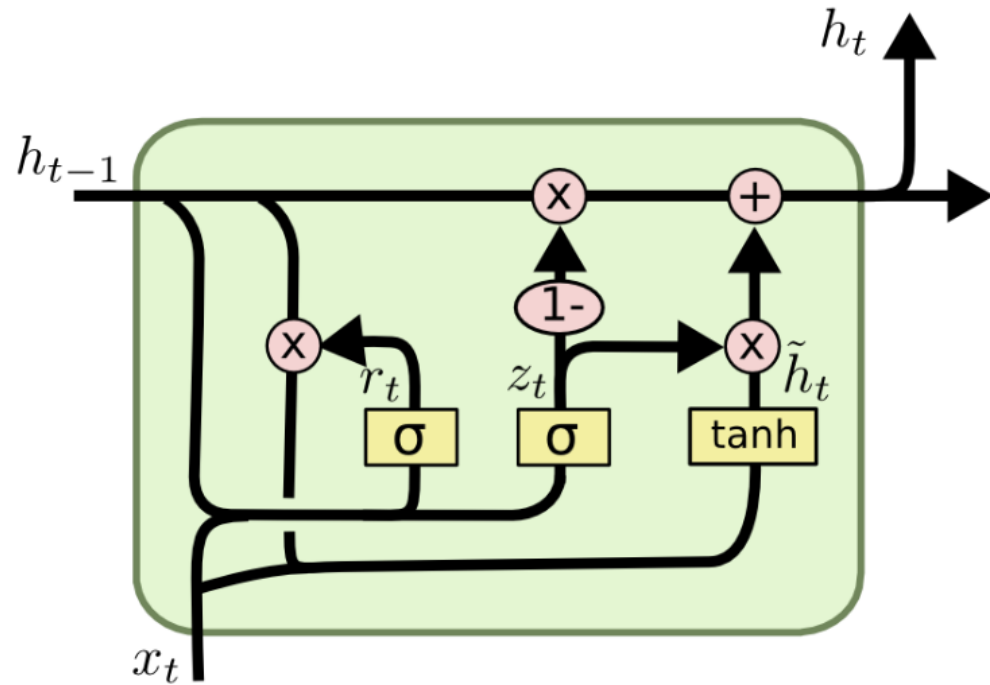


$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

- **output gate:** How much to expose
- Update the hidden state

Case Study: Gated Recurrent Unit (GRU)



$$z_t = \sigma (W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma (W_r \cdot [h_{t-1}, x_t])$$

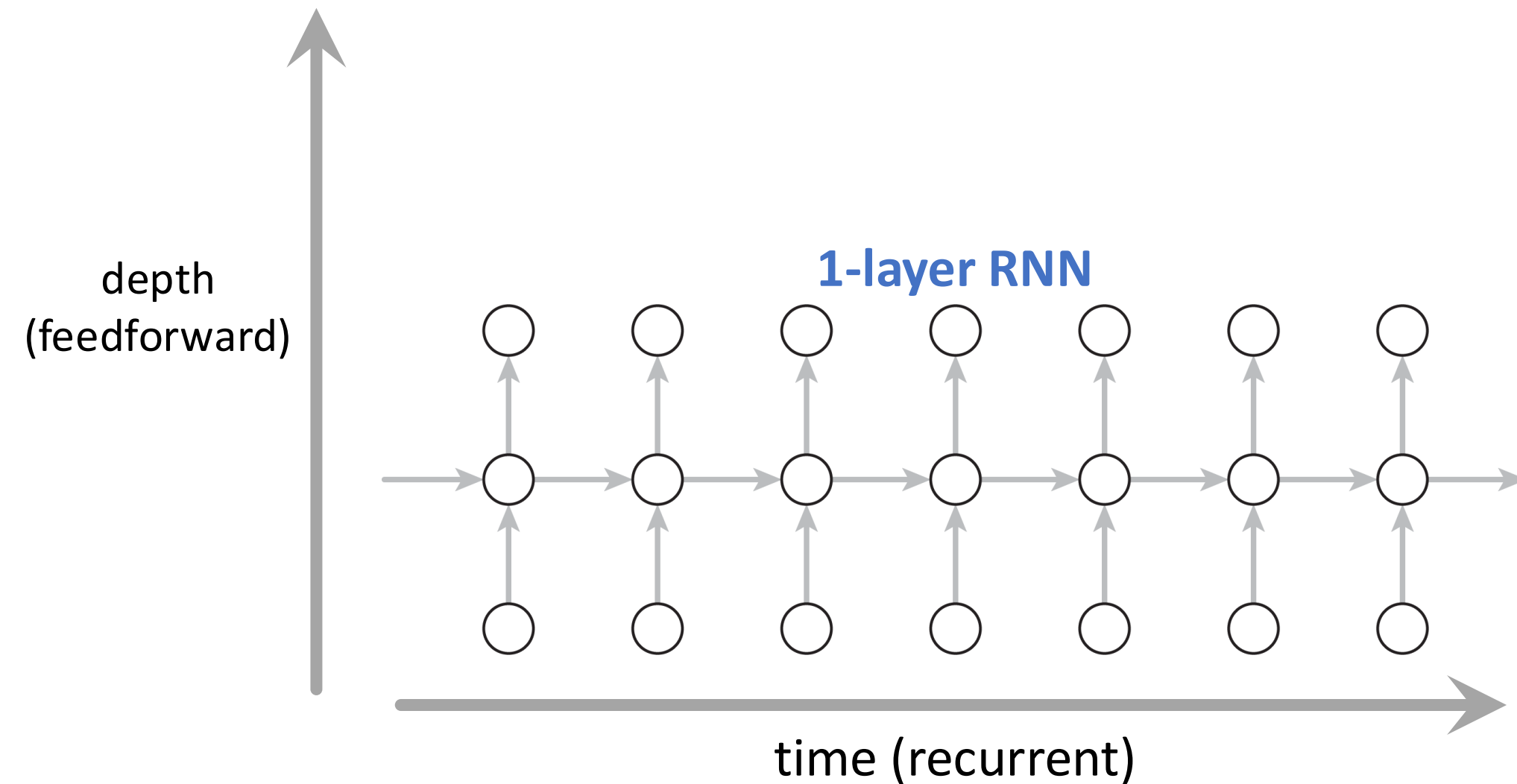
$$\tilde{h}_t = \tanh (W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Case Study: LSTM and GRU

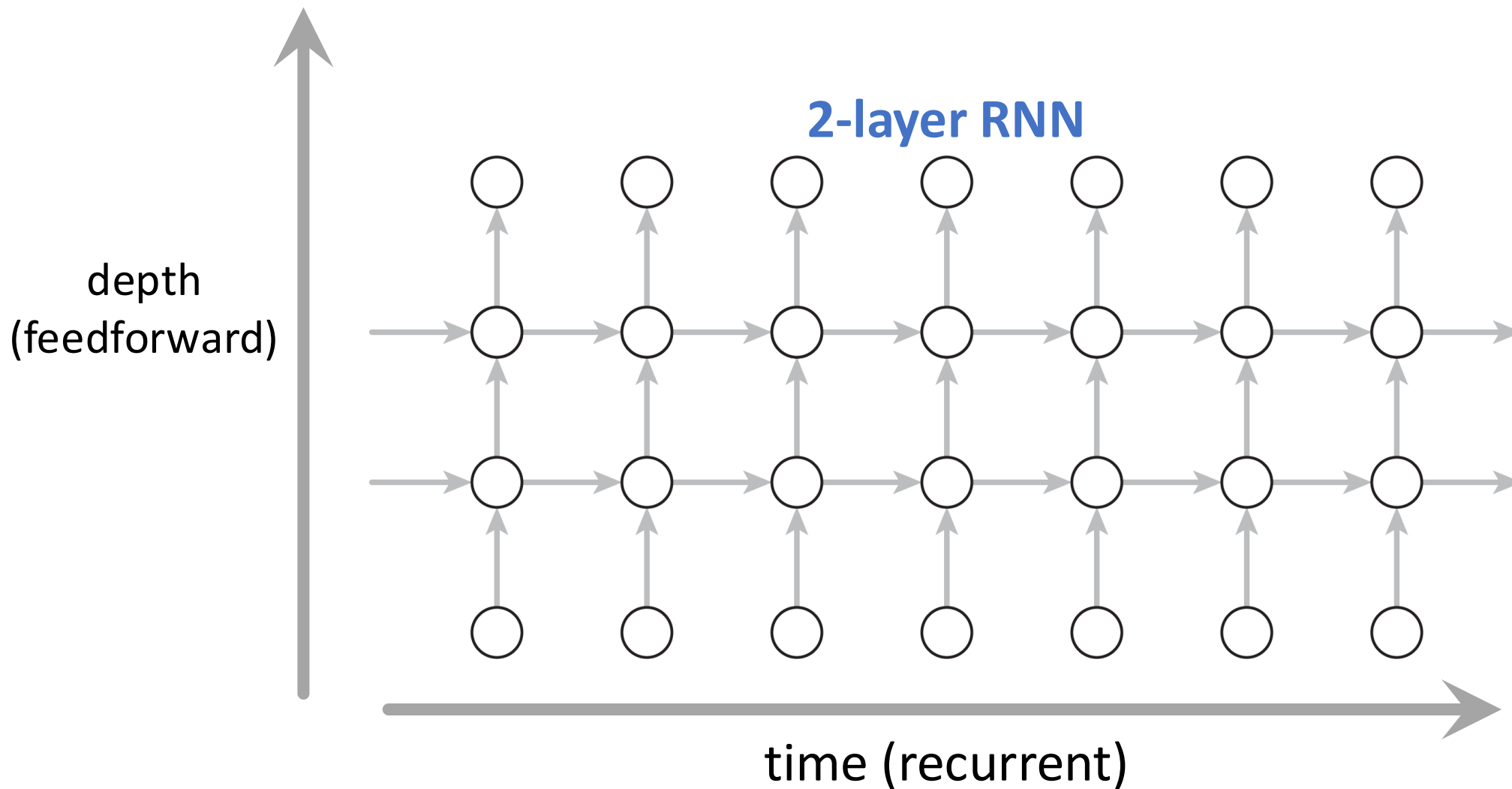
- Mitigate vanishing gradient
- Historically, the go-to architectures for RNN models
- Compress the past into a fixed-size state
- Transformer offers a more effective solution

Deep RNN



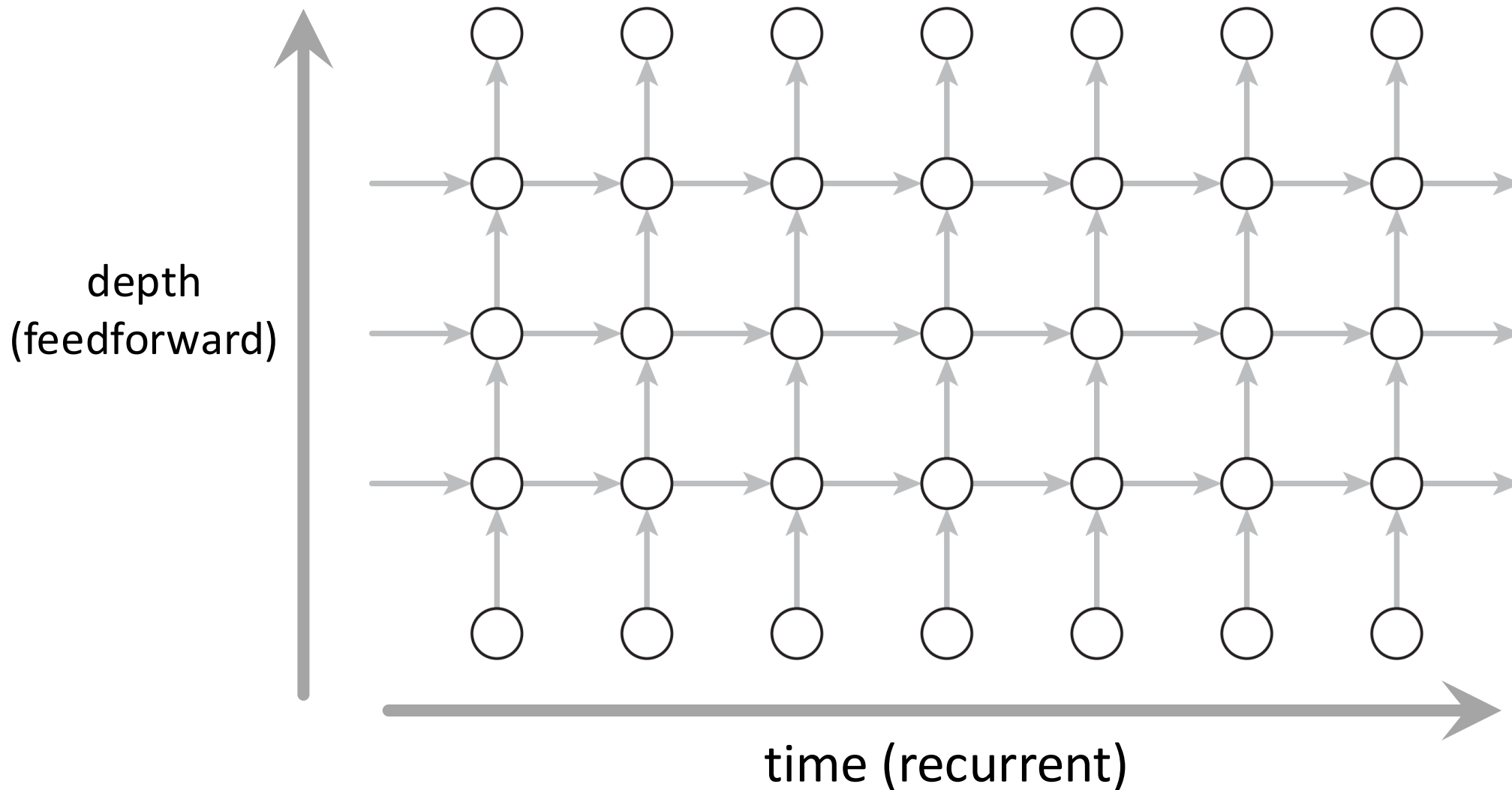
Deep RNN

2-layer RNN



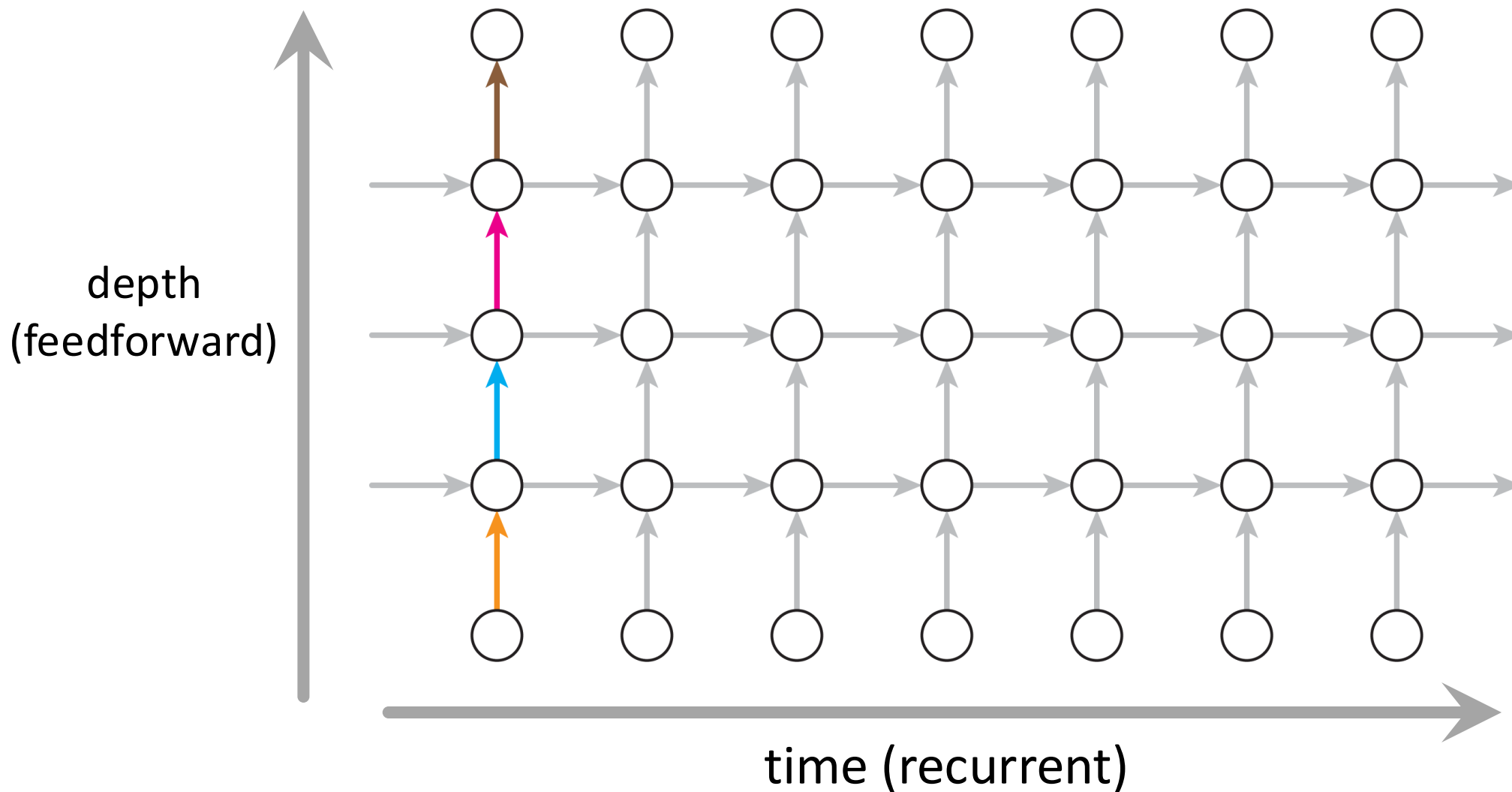
Deep RNN

3-layer RNN



Deep RNN

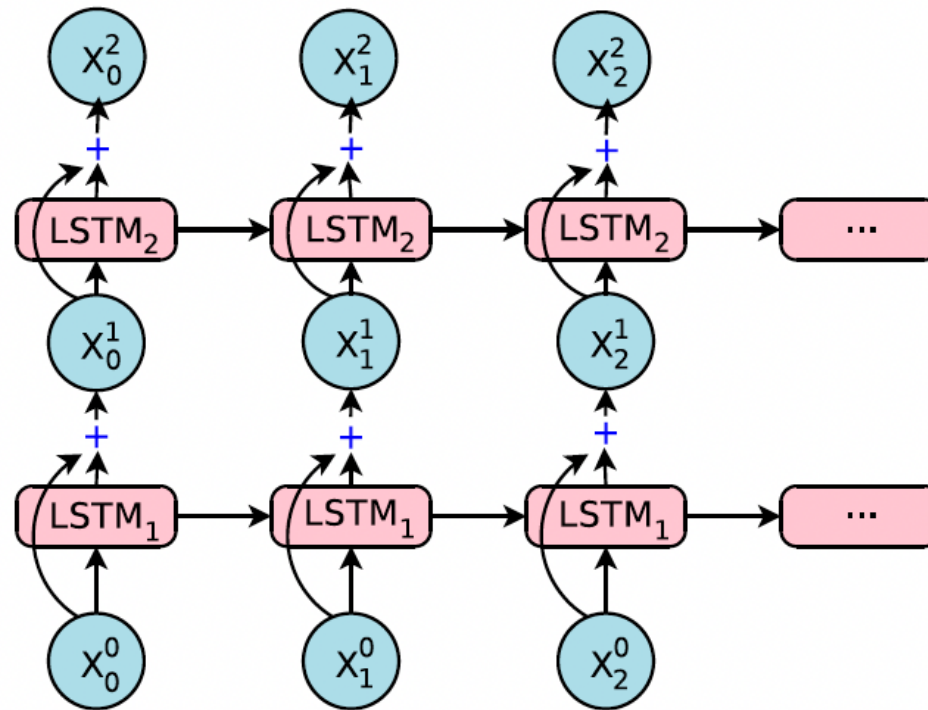
going **deep** in this direction



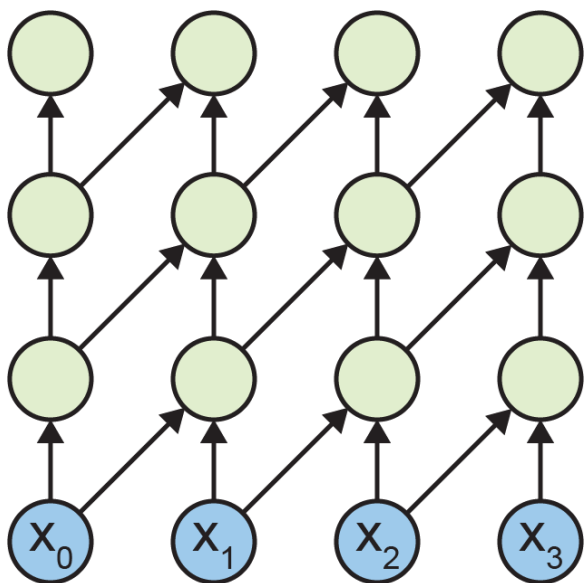
Example: Google's Neural Machine Translation System 2016

Deep RNN with LSTM units

- 16 layers enabled w/ residual connections
- degrade >4 layers w/o residual connections

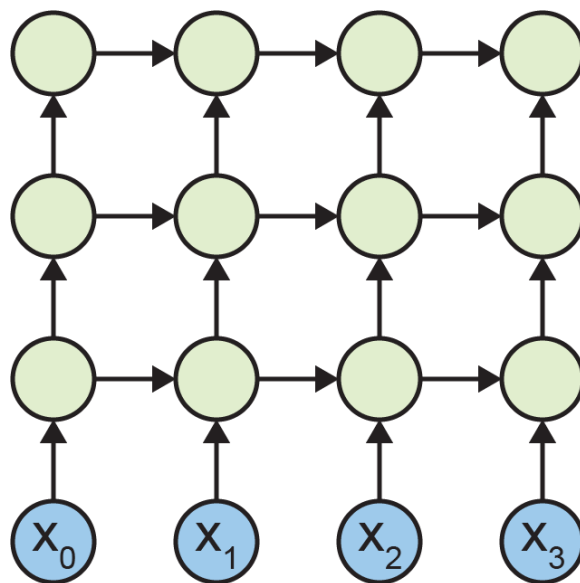


Paradigms of Sequence Modeling



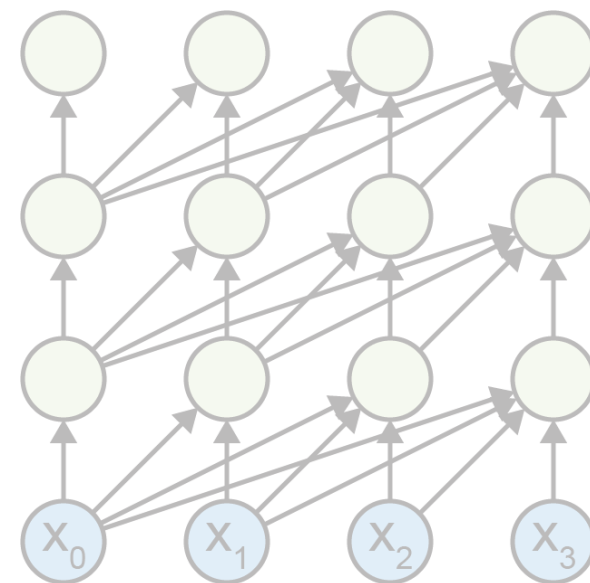
CNN

- parallelizable within a layer
- context limited by depth



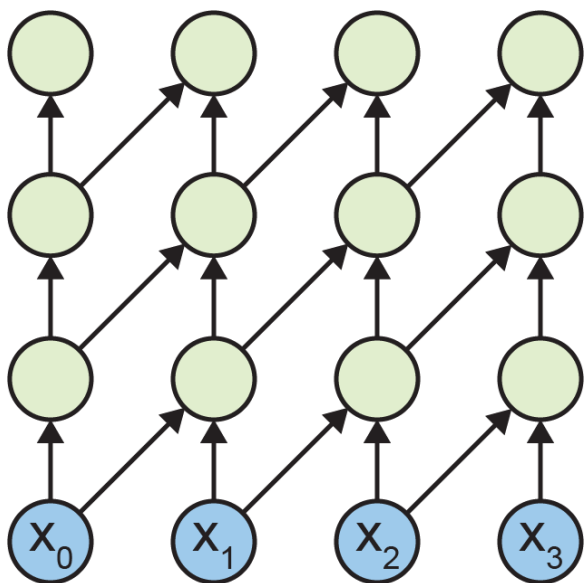
RNN

- not parallelizable within a layer
- long context



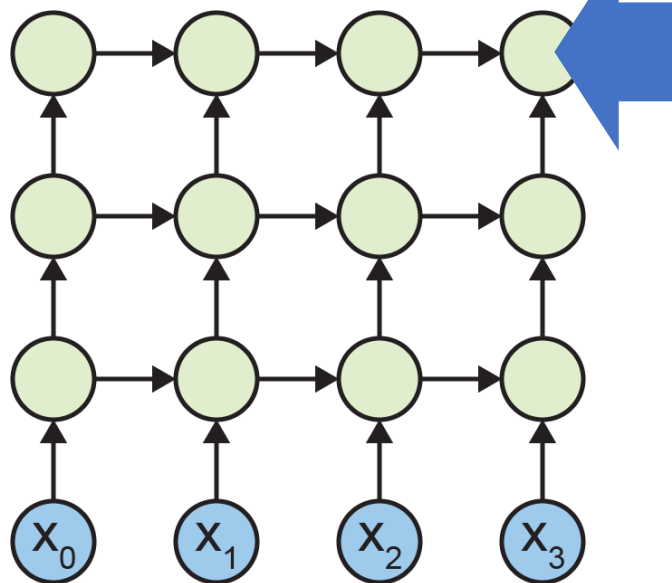
Attention

Paradigms of Sequence Modeling



CNN

- parallelizable within a layer
- context limited by depth

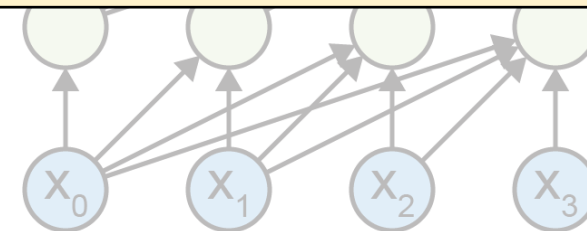


RNN

- not parallelizable within a layer
- long context

Step at time t must wait for all previous steps.

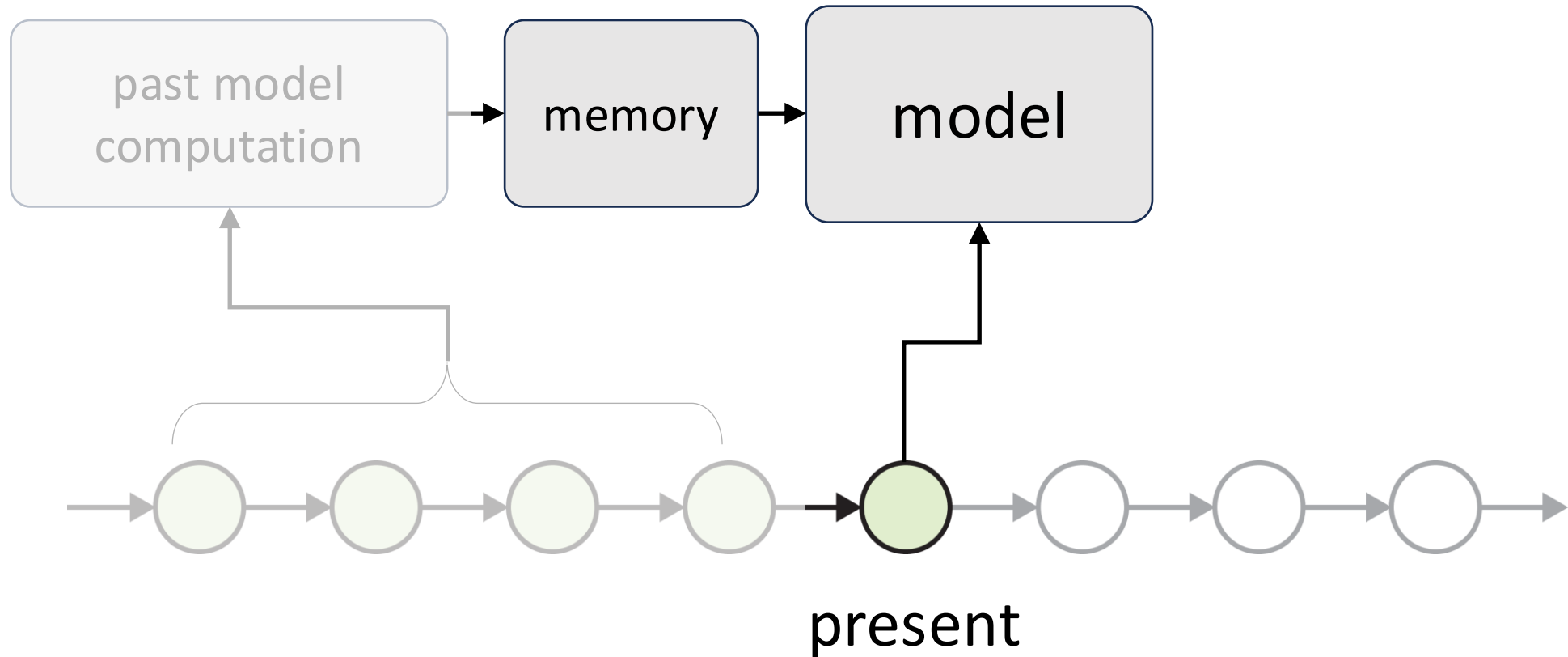
Not friendly to GPUs.



Attention

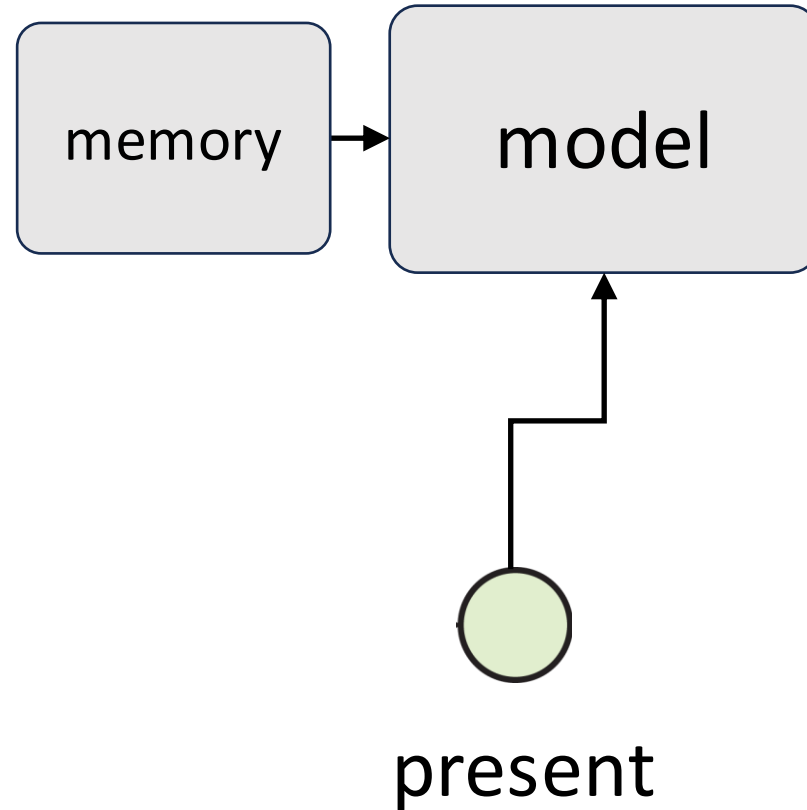
Memory Mechanisms

[Recap]: A General View of Sequence Modeling



- The model only sees present & memory

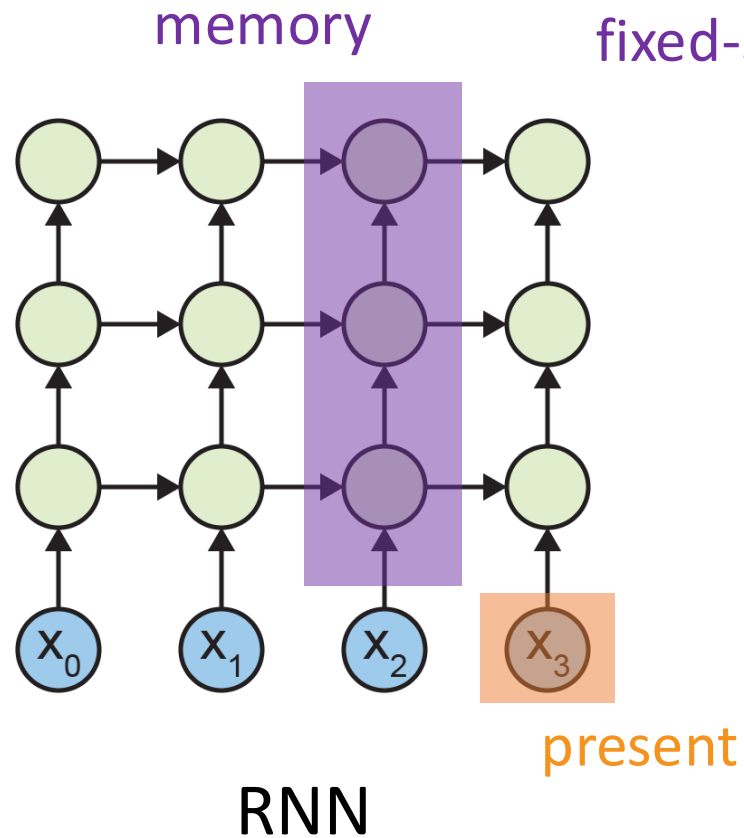
[Recap]: A General View of Sequence Modeling



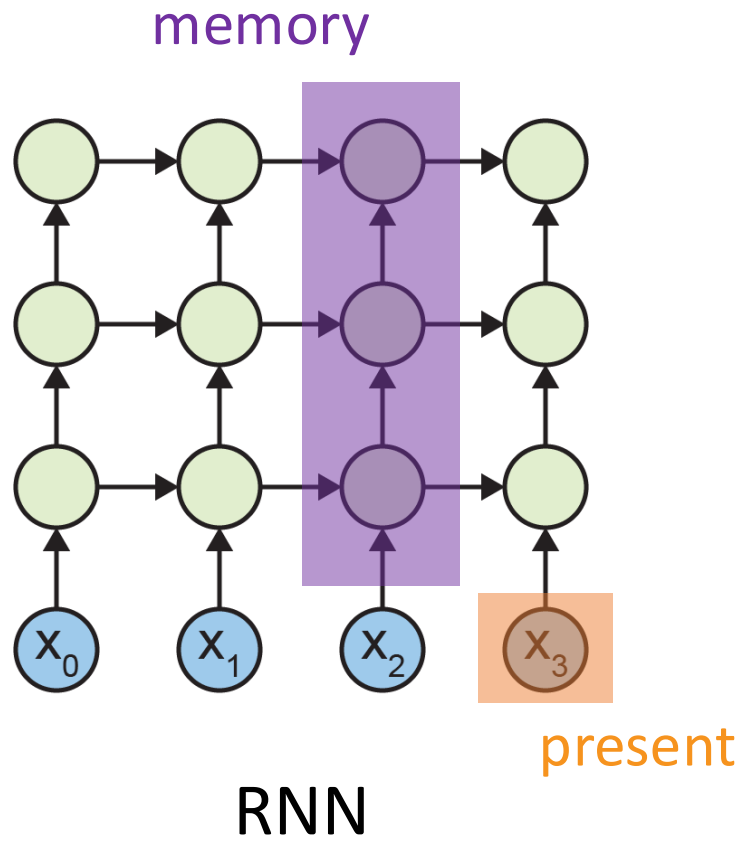
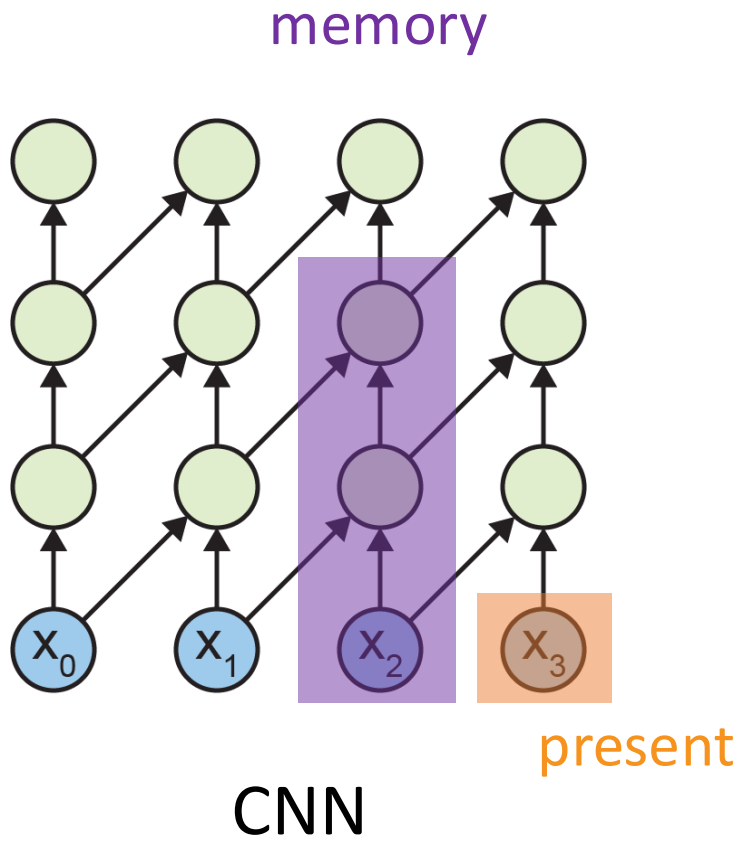
- The model only sees present & memory

What Serves as Memory?

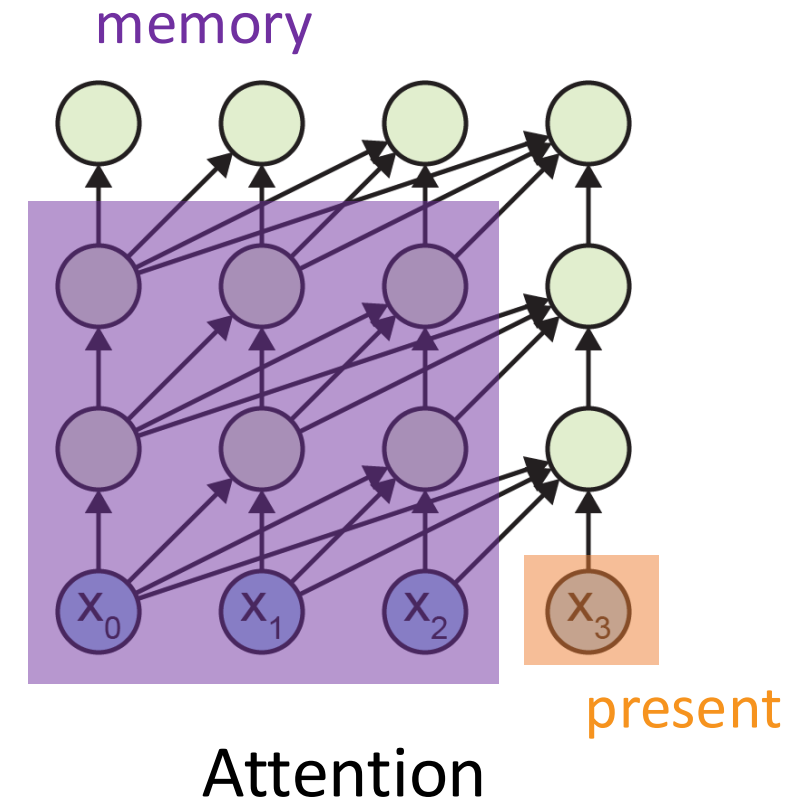
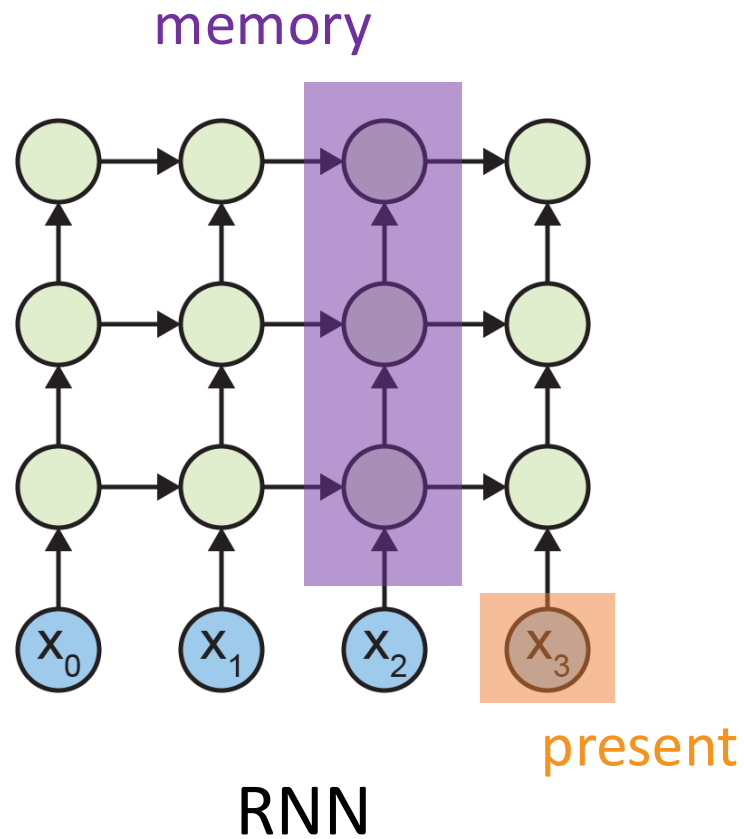
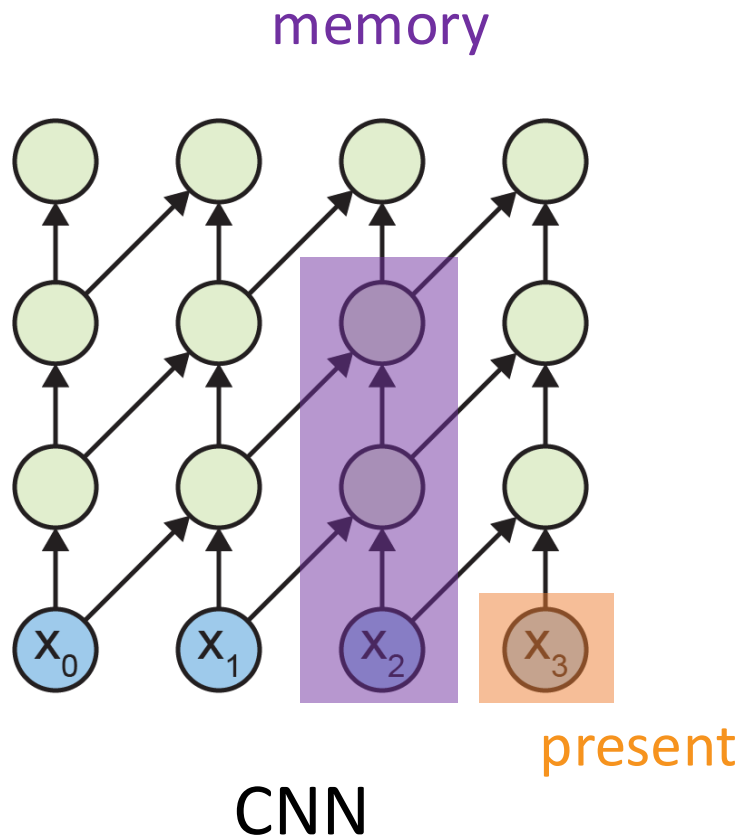
- The past is summarized in a fixed-size set of hidden states.



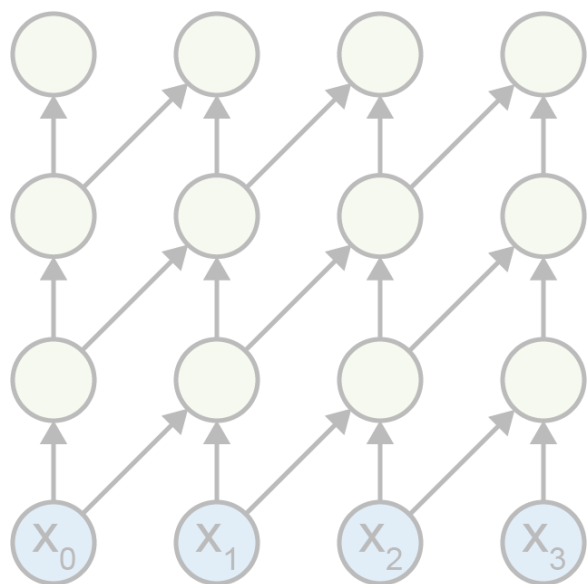
What Serves as Memory?



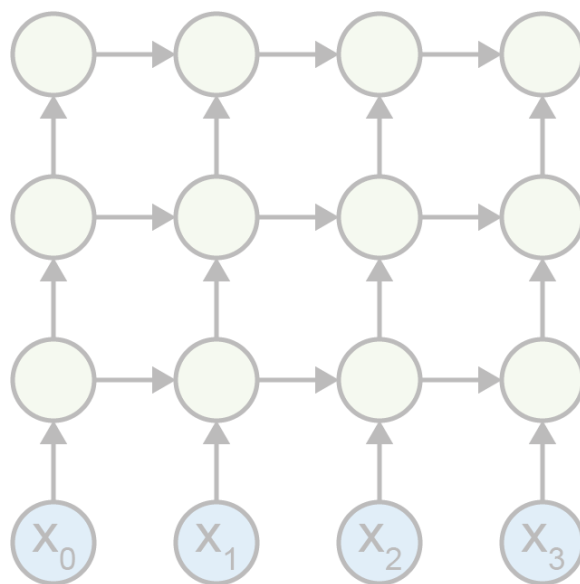
What Serves as Memory?



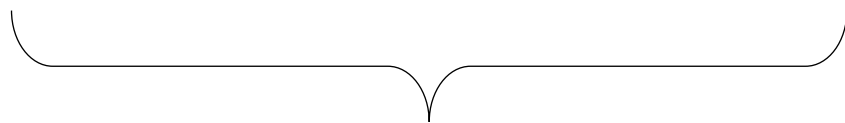
Summary



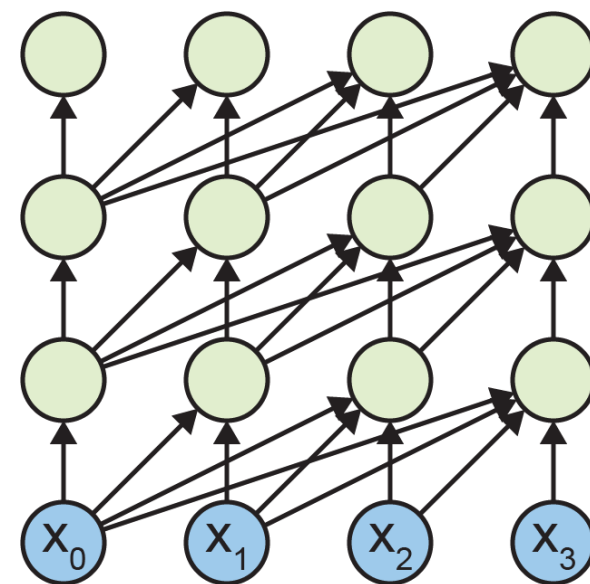
CNN



RNN



This lecture



Attention

Next lecture